내 컴퓨터에 개발자 '작업실' 만들기

미션 1 · 초등학생도 이해하는 쉬운 설명판

터미널 · Docker · Git/GitHub를 직접 해 보고,
모든 순간을 사진(증거)으로 남겼어요.

항목	내용
제출 저장소	https://github.com/80gina/E1-1
브랜치	main
사용한 컴퓨터	Windows 11 · PowerShell · Docker Desktop(WSL2)
Docker / Git 버전	Docker 29.9.2 / Git 2.53.0
증거 사진	모두 57장 (이 문서에 전부 실려 있어요)

> 이 문서는 **README.md(원본 기록)**를 어려운 말 없이 다시 풀어 쓴 **쉬운 설명판**이에요. 원본 README는 저장소에서 그대로 볼 수 있어요.

0. 이 문서 읽는 법

이 문서는 **'무엇을 했는지'**와 **'정말 했다는 증거(사진)'**를 짝지어 보여 줘요. 각 장은 이렇게 생겼어요.

- **쉬운 설명** : 어떤 일을 했고, 왜 했는지 비유로 말해 줘요.
- **명령어** : 컴퓨터에게 실제로 입력한 글자예요. (회색 상자)
- **그림** : 그때 찍은 화면이에요. 그림 이름의 번호(예: 21_docker_build)가 증거 번호예요.

> **그림 번호 약속** : 번호는 **한 일의 순서**예요. 01번이 맨 처음, 57번이 거의 마지막이에요. 맨 뒤 **부록**에 57장의 목록이 있어요.

차례

- 1. 한눈에 보기 : 세 가지 도구 이야기
- 2. 터미널 : 글자로 시키는 리모컨
- 3. 권한 : 서랍 열쇠 규칙
- 4. Docker 준비 : 도시락 가게 문 열기
- 5. 첫 도시락 : hello-world와 ubuntu
- 6. 내 레시피 : Dockerfile과 이미지
- 7. 정문 번호 : 포트 매핑
- 8. 창문과 사물함 : 바인드 마운트, 볼륨
- 9. 일기장과 클라우드 : Git과 GitHub
- 10. 보너스 1~4 : Docker Compose와 환경 변수
- 11. 보너스 5 : SSH 열쇠와 자물쇠
- 12. 포트가 겹치면? 직접 일으켜 본 실험
- 13. 어려웠던 일과 해결 방법
- 14. 질문과 쉬운 대답
- 15. 평가 문항 찾아보기
- 부록. 증거 사진 57장 목록

1. 한눈에 보기 : 세 가지 도구 이야기

컴퓨터로 프로그램을 만들 때 가장 먼저 필요한 건 **'작업실'**이에요. 작업실에는 도구 세 가지가 있어요.

도구	쉬운 말	비유
터미널	글자를 입력해서 컴퓨터에게 심부름을 시키는 창	말로 시키는 **리모컨**
Docker	프로그램을 도시락처럼 포장해서 어느 컴퓨터에서나 똑같이 실행해 줘요	**도시락 가게**
Git	내 컴퓨터에서 무엇을 바꿨는지 기록해 줘요	책상 위 **일기장**
GitHub	그 일기장을 인터넷에 올려 두는 곳이에요	**클라우드 사물함**

왜 이런 걸 만들까요?

"내 컴퓨터에서는 되는데, 친구 컴퓨터에서는 왜 안 되지?" 이런 일을 없애려고요. Docker로 포장하면 누구의 컴퓨터에서나 **똑같이** 움직여요.

내가 해낸 일

번호	해야 하는 일	결과	자세히 보기
1	터미널 기본 조작	완료	2장
2	파일 권한 바꾸기	완료	3장
3	Docker 설치와 점검	완료	4장
4	hello-world, ubuntu 실행	완료	5장
5	내 이미지 만들기(Dockerfile)	완료	6장
6	포트 매핑으로 접속	완료	7장
7	바인드 마운트와 볼륨	완료	8장
8	Git 설정과 GitHub 연결	완료	9장
보너스	Compose, 환경 변수, SSH 키	5개 모두 완료	10~11장
어려웠던 일	오류를 만나 해결하기	6가지 + 직접 일으킨 실험 1가지	12~13장

2. 터미널 : 글자로 시키는 리모컨

터미널은 마우스 대신 **글자**로 컴퓨터에게 일을 시켜요. 폴더(서랍)를 만들고, 파일(종이)을 만들고, 옮기고, 지웠어요.

내가 입력한 말	컴퓨터가 한 일	비유
pwd	지금 있는 곳을 알려 줘요	"나 지금 어디야?"
ls -Force	안에 든 것을 모두 보여 줘요 (숨은 것까지)	서랍 속 구경하기
mkdir practice	폴더를 만들어요	서랍 하나 새로 만들기
New-Item a.txt	빈 파일을 만들어요	빈 종이 한 장
cp / mv / rm	복사 / 이동·이름 바꾸기 / 지우기	복사기 / 옮기기 / 휴지통

```
pwd              # 지금 어디?
ls -Force        # 안에 뭐가 있나?
mkdir practice     # 폴더 만들기
New-Item a.txt     # 빈 파일 만들기
cp b.txt c.txt   # 복사
mv c.txt d.txt     # 이름 바꾸기
rm a.txt         # 지우기
```

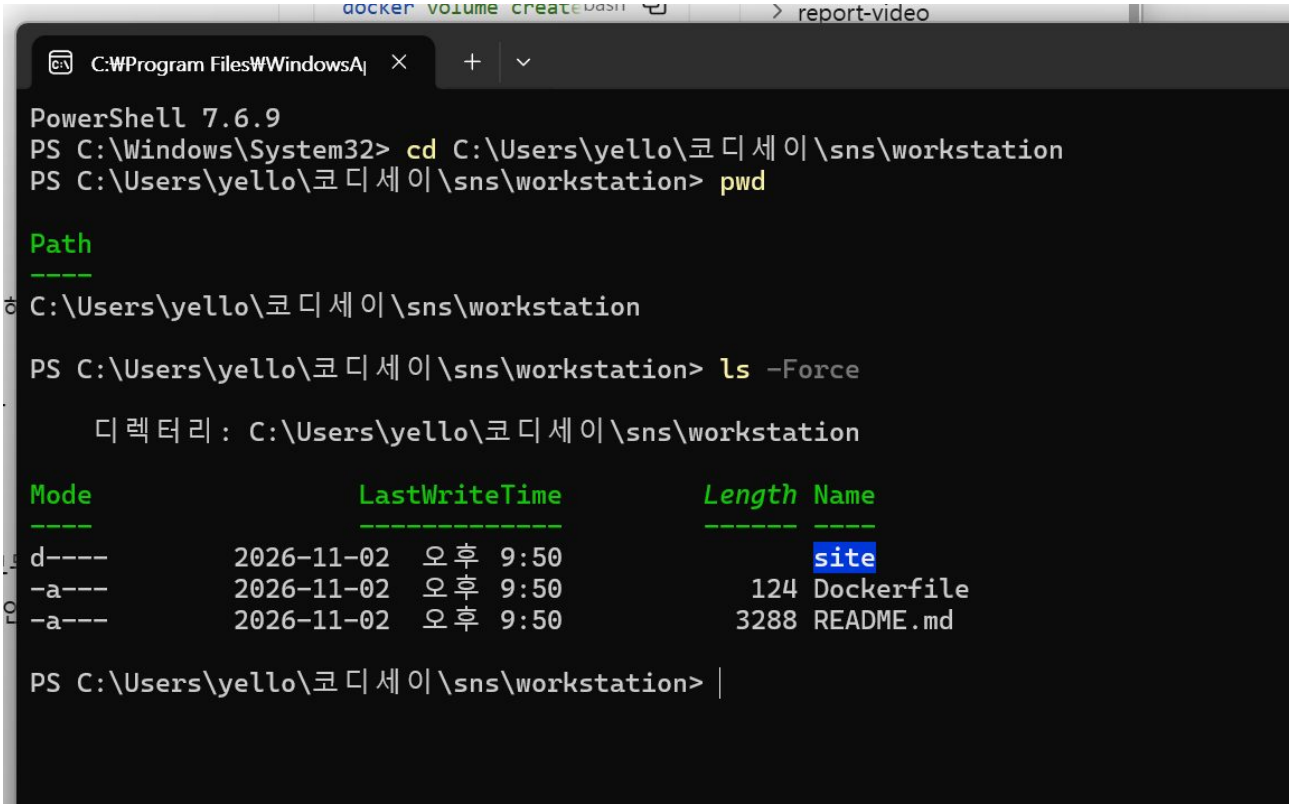


그림 01_pwd_ls : **지금 위치(pwd)와 안에 든 것(ls -Force)**을 확인했어요. workstation 폴더에 Dockerfile, README, site가 보여요.

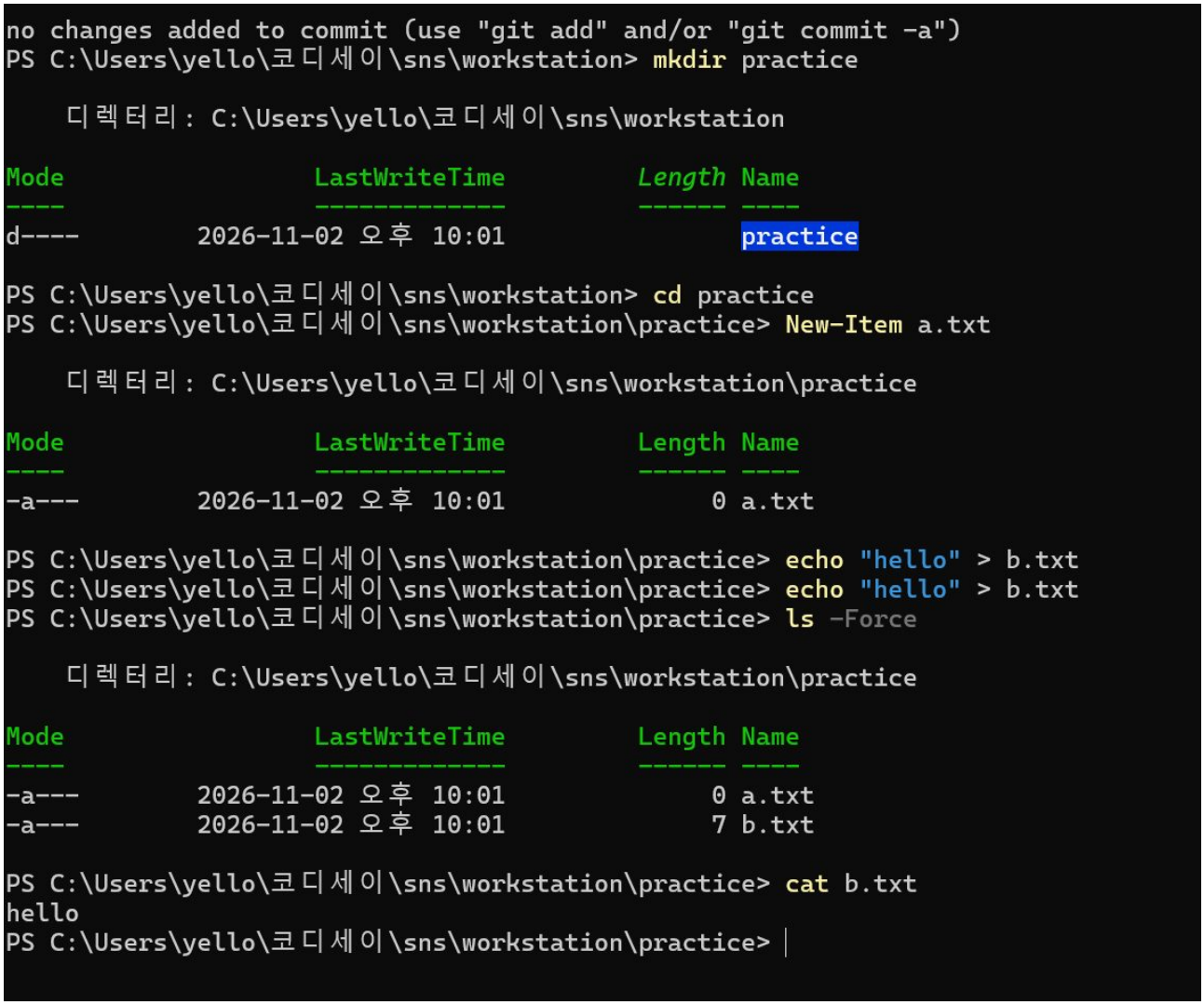


그림 03_create_files : **빈 파일(a.txt)과 글이 든 파일(b.txt)**을 만들었어요. cat으로 열어 보니 hello가 나와요.

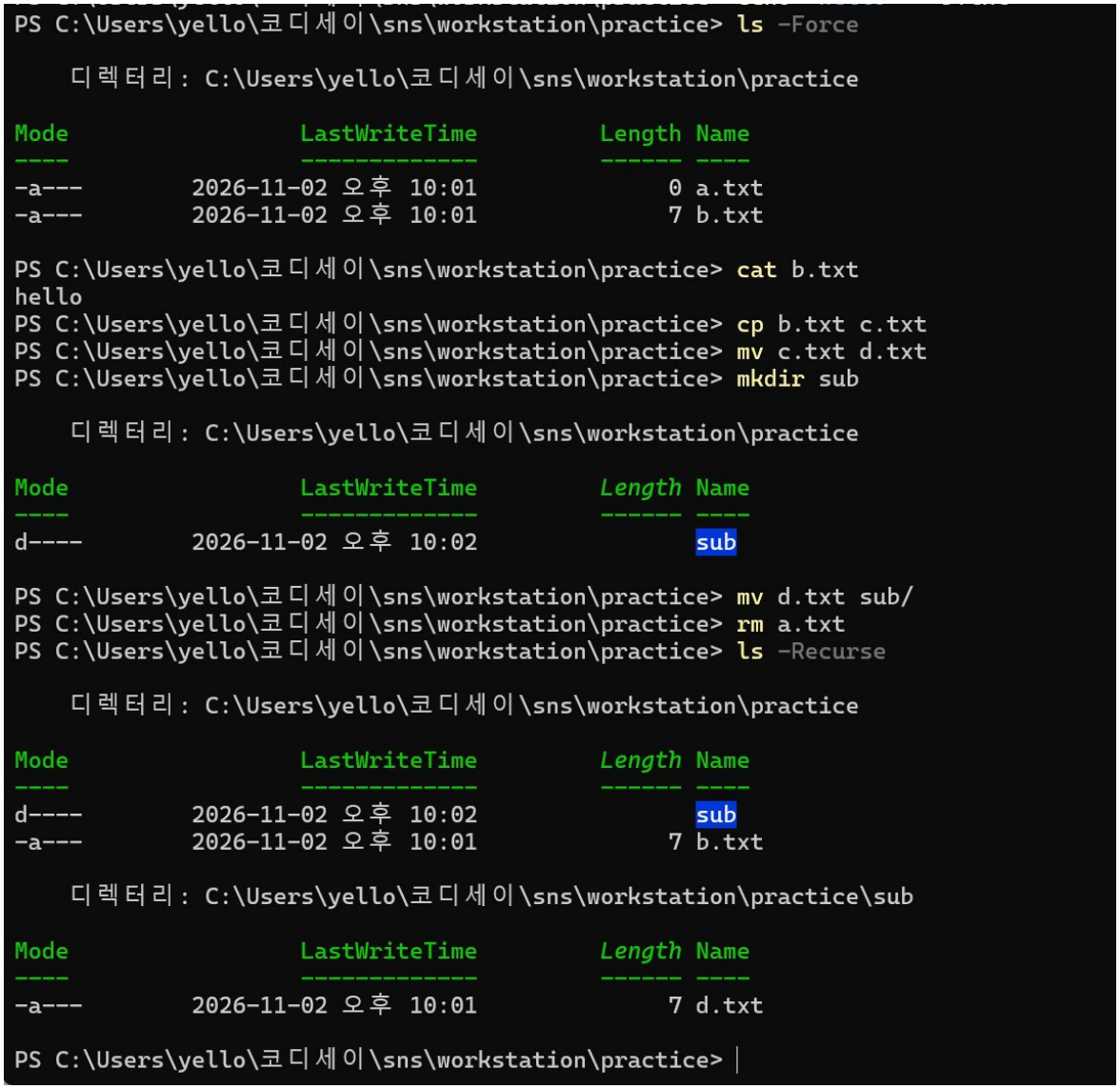


그림 04_copy_move_delete : **복사, 이름 바꾸기, 폴더로 옮기기, 지우기**를 했어요. 마지막 목록에 b.txt와 sub₩d.txt만 남았어요.

3. 권한 : 서랍 열쇠 규칙

파일과 폴더에는 **"누가 무엇을 할 수 있는지"** 규칙이 있어요. 그 규칙을 바꿔 보고, 바뀌기 **전과 후**를 비교했어요.

글자	뜻	숫자
r	읽기 (볼 수 있어요)	4
w	쓰기 (고칠 수 있어요)	2
x	실행 (작동시킬 수 있어요)	1

숫자 세 개는 **주인 / 같은 모둠 / 다른 사람** 순서예요. 읽기+쓰기+실행을 더해서 써요.

표기	계산	모양	쓰임
755	(4+2+1) (4+1) (4+1)	rwxr-xr-x	주인은 전부 가능, 나머지는 보고 실행만. **폴더**나 실행 파일
644	(4+2) 4 4	rw-r--r--	주인만 고칠 수 있고, 나머지는 읽기만. 보통 **파일**
700	(4+2+1) 0 0	rwx------	나만 쓰는 비밀 폴더

Windows에서 한 권한 실험

Windows에는 chmod가 없어서 **다른 도구**(attrib, icacls)로 같은 개념을 실험했어요.



그림 05_file_permission : 파일을 **'읽기 전용(R)'**으로 바꿨어요. 쓰려고 하면 **Access is denied**(거부)가 나와요. 권한이 진짜 작동한다는 증거예요.

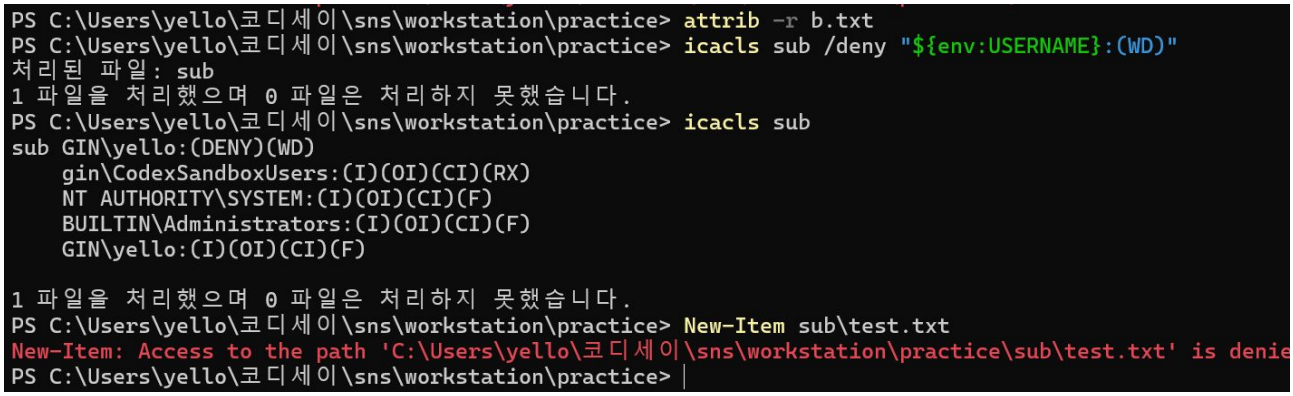


그림 06_dir_permission : 폴더에 **'쓰기 금지(DENY)'**를 걸었더니 새 파일을 만들 수 없어요.

```
1 파일을 처리했으며 0 파일은 처리하지 못했습니다.
PS C:\Users\yello\코디세이\sns\workstation\practice> New-Item sub\test.txt
New-Item: Access to the path 'C:\Users\yello\코디세이\sns\workstation\practice\sub\test.txt' is denie
PS C:\Users\yello\코디세이\sns\workstation\practice> icacls sub /remove:d "${env:USERNAME}"
처리된 파일: sub
1 파일을 처리했으며 0 파일은 처리하지 못했습니다.
PS C:\Users\yello\코디세이\sns\workstation\practice> icacls sub
sub gin\CodexSandboxUsers:(I)(OI)(CI)(RX)
    NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
    BUILTIN\Administrators:(I)(OI)(CI)(F)
    GIN\yello:(I)(OI)(CI)(F)

1 파일을 처리했으며 0 파일은 처리하지 못했습니다.
PS C:\Users\yello\코디세이\sns\workstation\practice>
```

그림 07_dir_restore : 금지를 풀어서 **원래대로** 돌려놓았어요. (DENY가 사라졌어요.)

리눅스(ubuntu 안)에서 한 진짜 chmod

```
chmod 644 f.txt     # 파일: rw-r--r--
chmod 700 d          # 폴더: rwx------
chmod 755 f.txt     # 파일: rwxr-xr-x
```

```
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-polici
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
root@fd38a5f12b56:/# cd /tmp
root@fd38a5f12b56:/tmp# echo "hello" > f.txt
root@fd38a5f12b56:/tmp# mkdir d
root@fd38a5f12b56:/tmp# ls -ld f.txt d
drwxr-xr-x 2 root root 4096 Nov  2 13:41 d
-rw-r--r-- 1 root root    6 Nov  2 13:41 f.txt
root@fd38a5f12b56:/tmp# chmod 644 f.txt
root@fd38a5f12b56:/tmp# chmod 700 d
root@fd38a5f12b56:/tmp# ls -ld f.txt d
drwx------ 2 root root 4096 Nov  2 13:41 d
-rw-r--r-- 1 root root    6 Nov  2 13:41 f.txt
root@fd38a5f12b56:/tmp# chmod 755 f.txt
root@fd38a5f12b56:/tmp# ls -l f.txt
-rwxr-xr-x 1 root root 6 Nov  2 13:41 f.txt
root@fd38a5f12b56:/tmp#
```

그림 17_chmod_linux : ubuntu 안에서 **chmod 644 / 700 / 755**를 하고 ls -l로 전과 후를 비교했어요. 폴더 d가 drwx------로, 파일이 -rwxr-xr-x로 바뀌었어요.

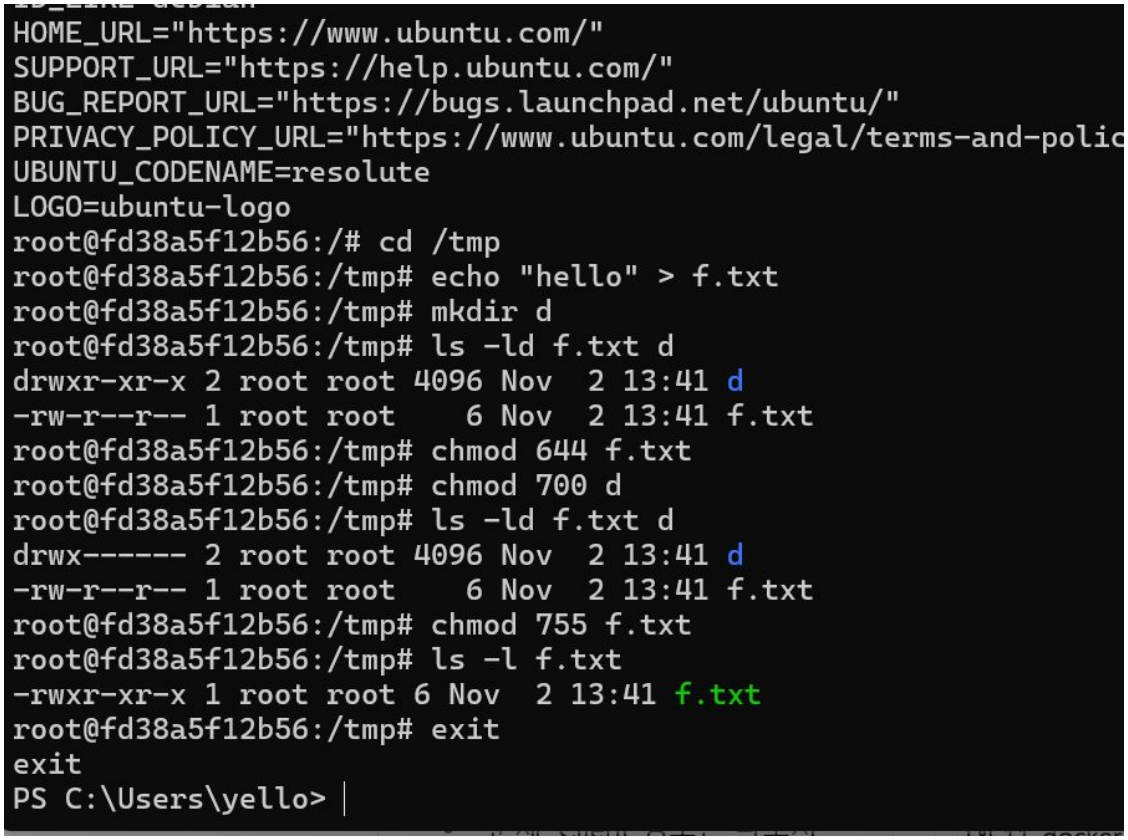


그림 17a_chmod_exit : exit를 치고 컨테이너에서 나와 Windows PowerShell로 돌아온 모습이에요.

4. Docker 준비 : 도시락 가게 문 열기

Docker는 프로그램을 **도시락처럼 포장**해요. 도시락 가게를 열려면 **설치**하고 **불을 켜야(엔진 실행)** 해요.

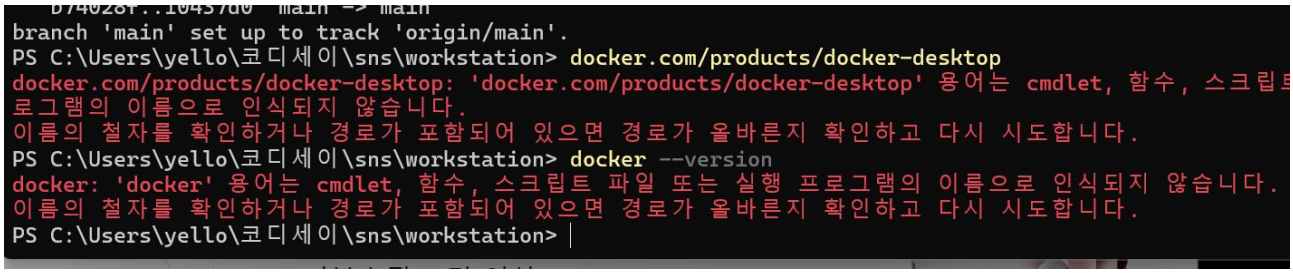


그림 09a_docker_before : 설치하기 **전**에는 docker라고 쳐도 "그런 명령은 몰라요"라고 나와요.

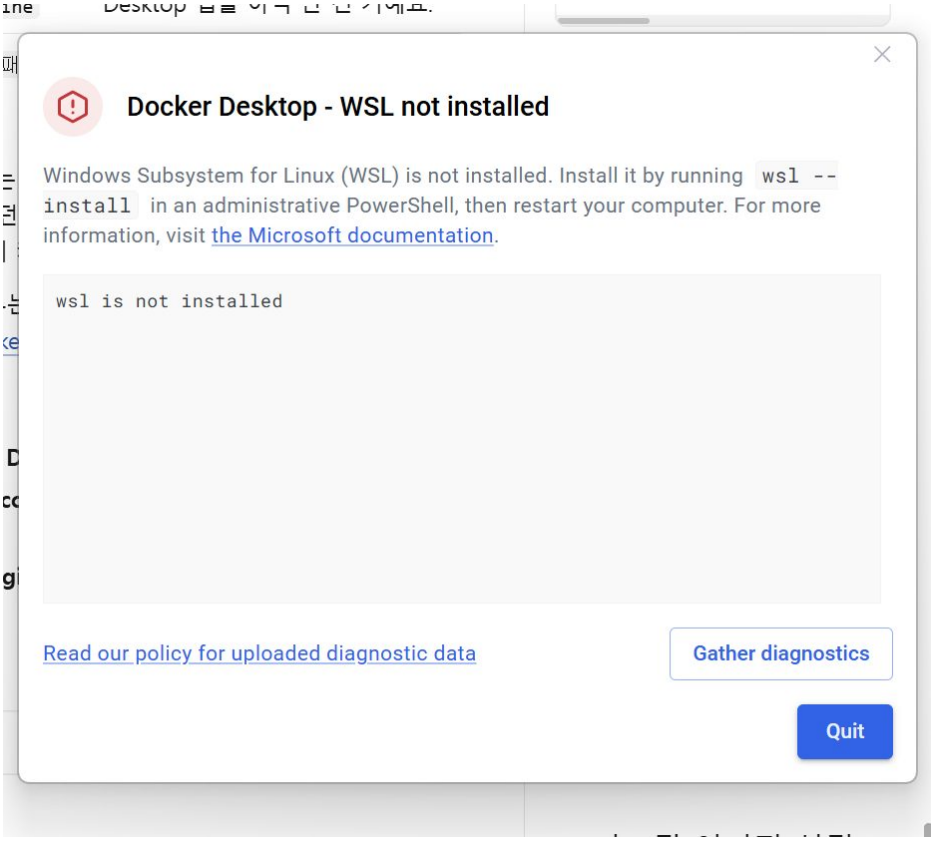


그림 12_wsl_not_installed : Docker Desktop을 켜자 **"WSL이 없어요"**라는 창이 떴어요. 도시락을 데울 **전자레인지(WSL)**가 없는 거예요.

```
wsl --install        # 관리자 PowerShell에서 입력한 설치 명령 (이후 재부팅)
```

```
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.2.9
    Path:     C:\Users\yello\.docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.7.21
    Path:     C:\Users\yello\.docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.3.2
    Path:     C:\Users\yello\.docker\cli-plugins\docker-pass.exe
  sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
    Version:  v0.13.0
    Path:     C:\Users\yello\.docker\cli-plugins\docker-sandbox.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.27.0
    Path:     C:\Users\yello\.docker\cli-plugins\docker-scout.exe

Server:
failed to connect to the docker API at npipe:////./pipe/docker_engine; check if the path
 running: open //./pipe/docker_engine: The system cannot find the file specified.
PS C:\Users\yello> winget install -e --id Docker.DockerDesktop
'msstore' 원본을 사용하려면 다음 계약을 확인해야 합니다.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
원본이 제대로 작동하려면 현재 컴퓨터의 두 글자 지리적 지역을 백 엔드 서비스로 보내야 합니

모든 원본 사용 약관에 동의하십니까?
[Y] 예  [N] 아니요: y
이미 설치된 기존 패키지를 찾았습니다. 설치된 패키지를 업그레이드하는 중...
사용 가능한 업그레이드를 찾을 수 없습니다.
구성된 원본에서 사용할 수 있는 최신 패키지 버전이 없습니다.
PS C:\Users\yello>
```

그림 10_docker_engine_off : docker info를 쳤더니 Client는 나오는데 **Server는 연결 실패**예요. 가게 불(엔진)이 꺼져 있었어요.

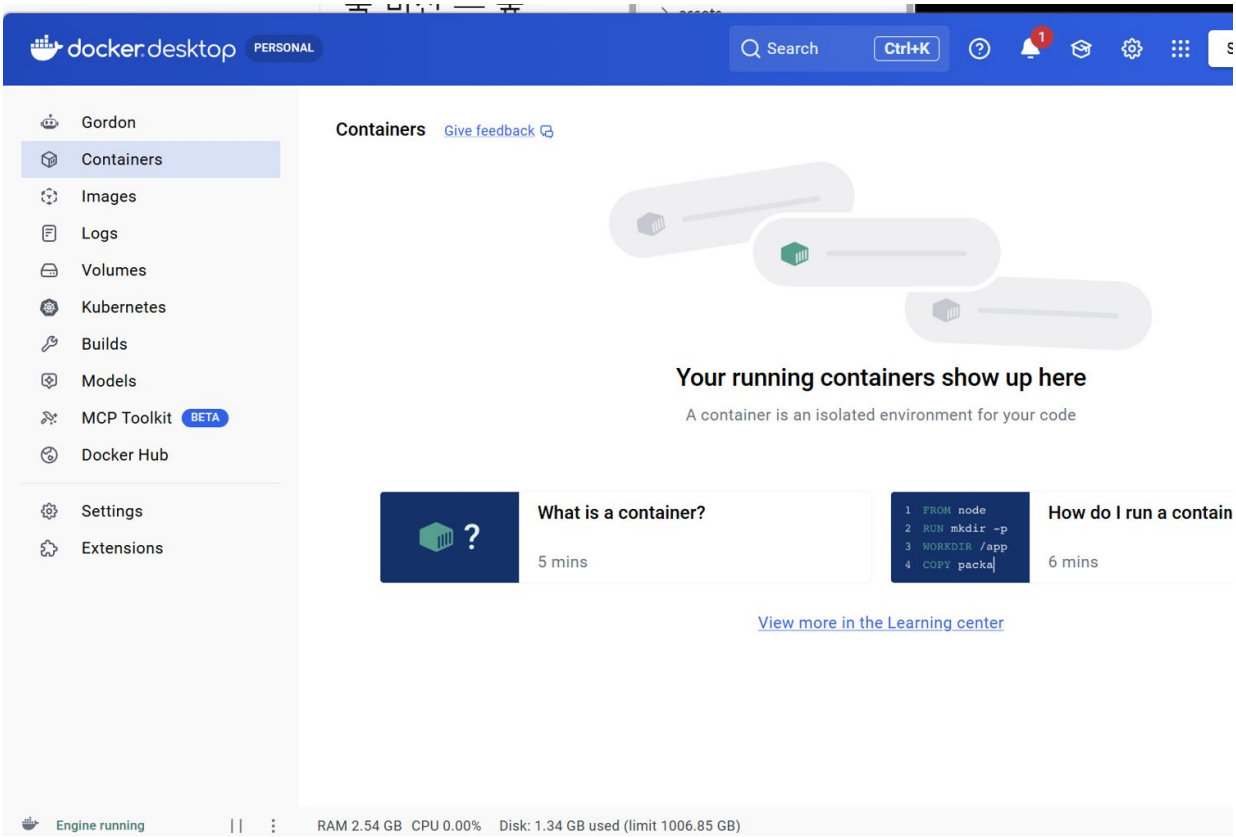


그림 11a_docker_desktop_running : Docker Desktop을 켜자 왼쪽 아래에 **Engine running**(엔진 켜짐)이 보여요.

```
PS C:\Users\yello> docker --version
Docker version 29.9.2, build a82e528
PS C:\Users\yello>
```

그림 **11_docker_version** : **docker --version** : Docker version 29.9.2

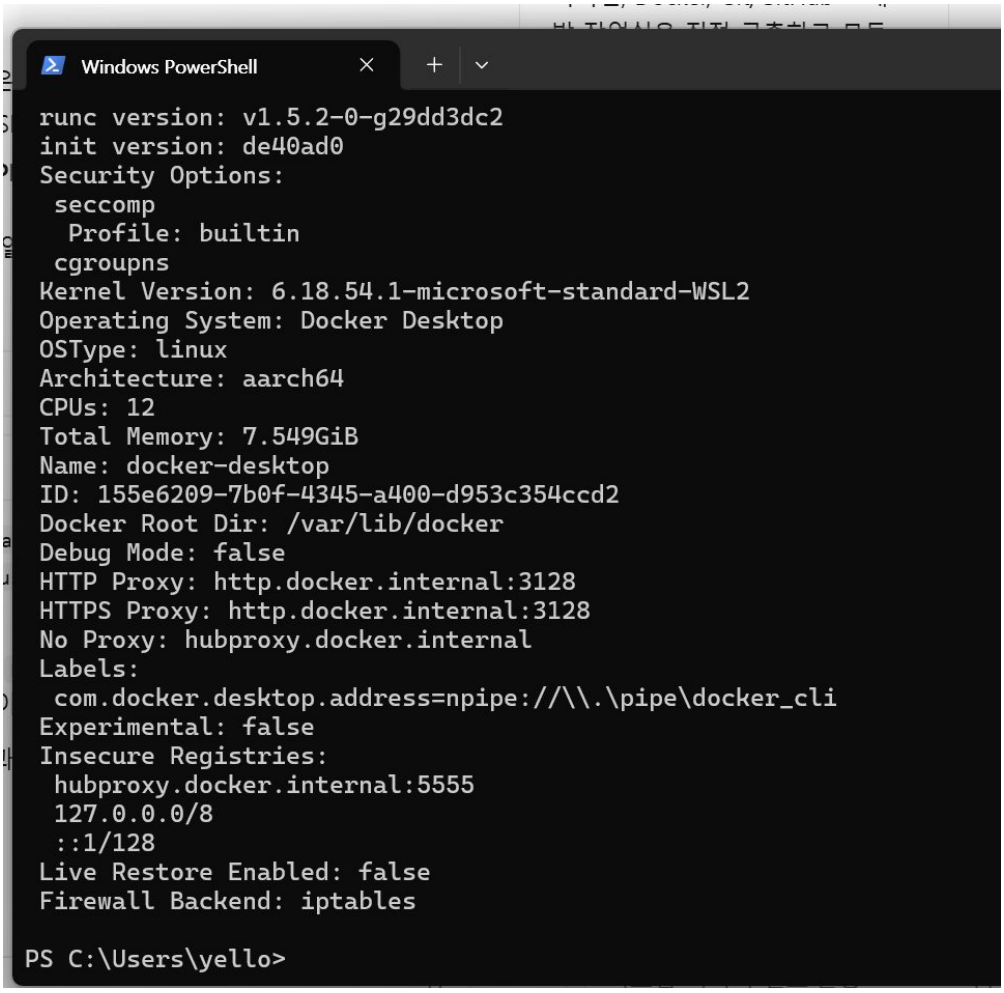


그림 **11_docker_info_ok** : **docker info** : 이제 Server 정보(리눅스, WSL2 등)가 나와요. 엔진이 켜졌다는 증거예요.

5. 첫 도시락 : hello-world와 ubuntu

hello-world : 가장 작은 도시락

Docker가 잘 작동하는지 알려 주는 아주 작은 도시락이에요. **Hello from Docker!**가 나오면 성공이에요. 이 도시락은 인사만 하고 바로 끝나요.

```
docker run hello-world
docker images       # 도시락 레시피(이미지) 목록
docker ps -a        # 도시락(컨테이너) 목록
```

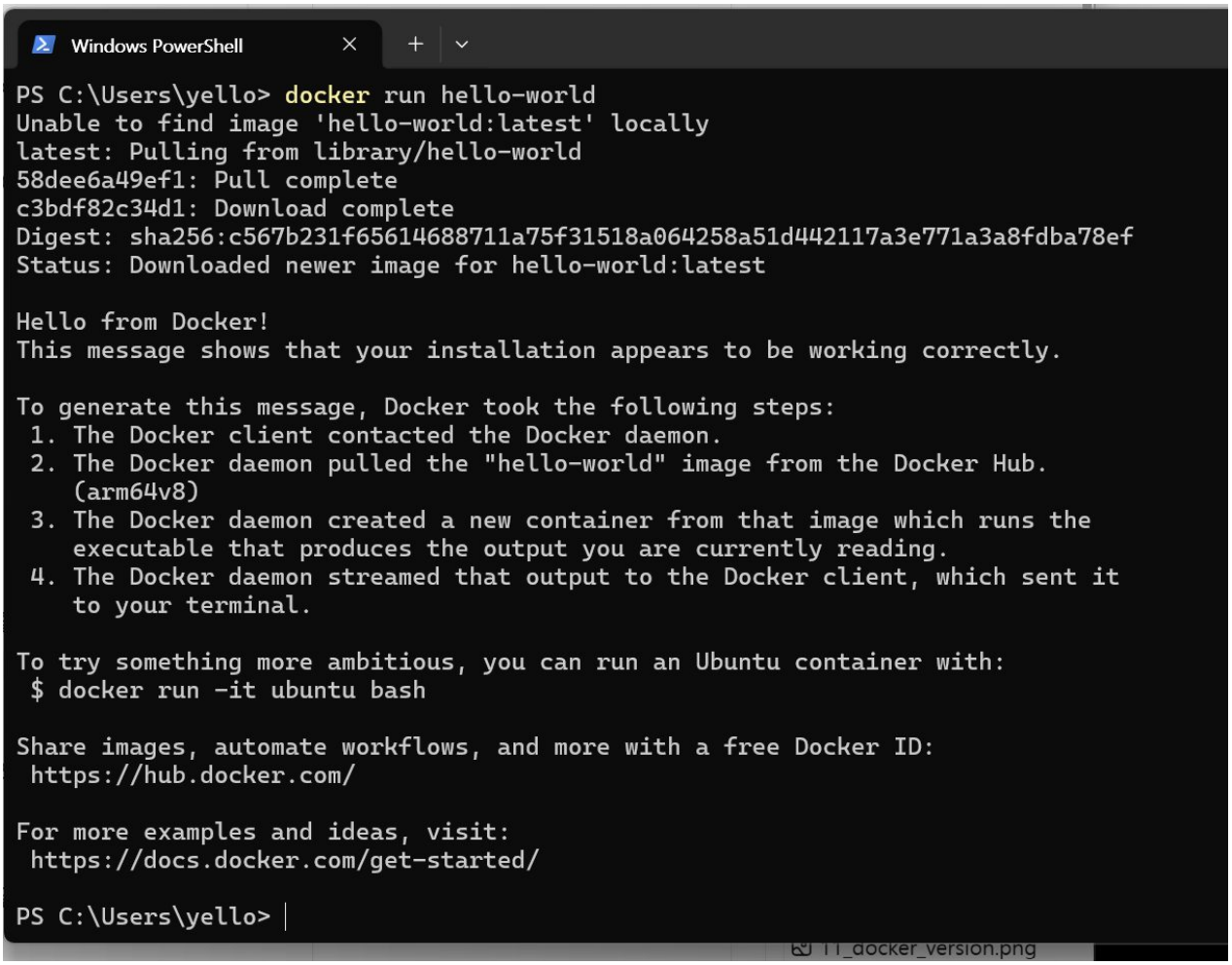


그림 14_hello_world : **Hello from Docker!** 성공! 처음 실행이라 이미지를 내려받는 과정(Pull complete)도 보여요.



그림 15_images_ps : docker images에는 hello-world 레시피가, docker ps -a에는 **Exited (0)**(오류 없이 끝남) 도시락이 보여요.

ubuntu : 안으로 들어가 보기

ubuntu는 리눅스예요. 도시락 **안으로 들어가서** ls, echo 같은 명령을 해 봤어요. 안에서는 화면 맨 앞이 **root@...**로 바뀌어요.

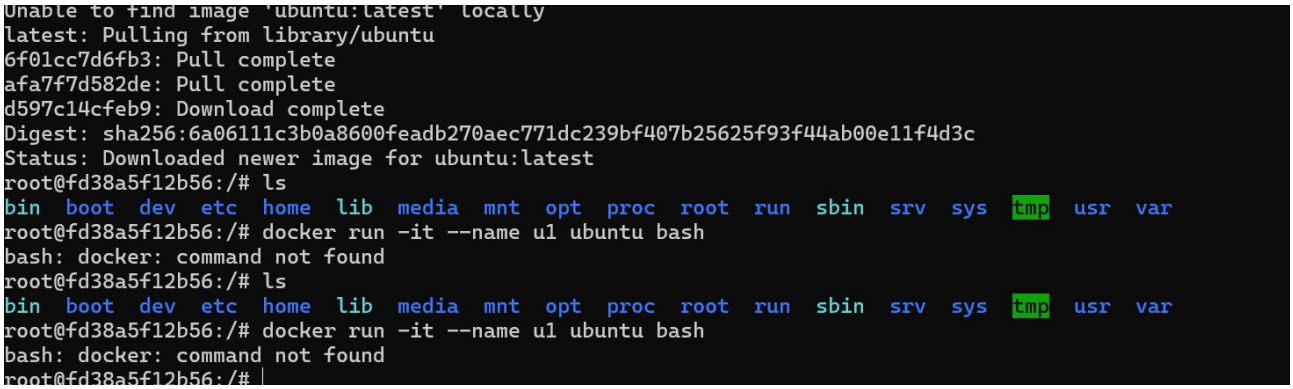


그림 16a_docker_inside_container_error : 도시락 **안에서** docker라고 쳤더니 **command not found**가 나왔어요. 안에는 docker가 없기 때문이에요. 바깥(Windows)에서 쳐야 해요.

들어가는 방법 두 가지, 나올 때 달라요

방법	하는 일	exit 하면?
run -it	새 도시락을 만들어서 들어가요	도시락이 **끝나요** (Exited)
exec	이미 켜져 있는 도시락에 들어가요	도시락이 **계속 켜져 있어요** (Up)

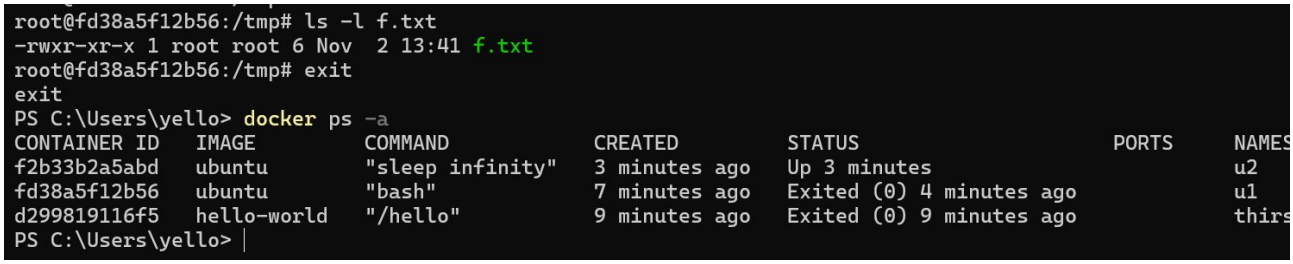


그림 18_ps_logs : docker ps -a : **u2는 Up(켜짐), u1과 hello-world는 Exited(끝남)**. 두 방법의 차이가 한눈에 보여요.

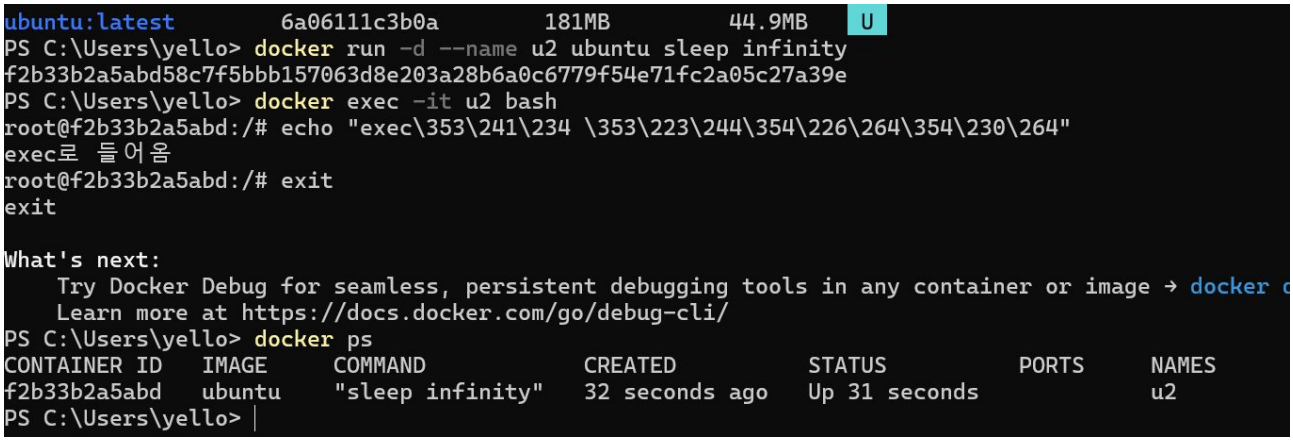


그림 19_exec_vs_run : **exec**로 u2에 들어갔다 나와도 docker ps에 u2가 **Up**으로 남아 있어요.

```
root@fd38a5f12b56:/tmp# ls -l f.txt
-rwxr-xr-x 1 root root 6 Nov  2 13:41 f.txt
root@fd38a5f12b56:/tmp# exit
exit
PS C:\Users\yello> docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   c567b231f656       22.6kB         10.3kB    U
ubuntu:latest        6a06111c3b0a        181MB          44.9MB    U
PS C:\Users\yello>
```

그림 **18a_images_after_exit** : 나온 뒤 docker images : hello-world와 ubuntu 두 레시피가 보여요.

가게 상태 살피기

```
docker logs u1              # 도시락이 남긴 기록 보기
docker stats --no-stream    # 힘(CPU)·기억(메모리) 얼마나 쓰나?
```

```
PS C:\Users\yello> docker ps
CONTAINER ID   IMAGE     COMMAND            CREATED          STATUS          PORTS     NAMES
f2b33b2a5abd   ubuntu    "sleep infinity"   32 seconds ago   Up 31 seconds             u2
PS C:\Users\yello> docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O    PIDS
f2b33b2a5abd   u2        0.00%     3.434MiB / 7.549GiB   0.04%     1.17kB / 126B   532kB / 0B   1
PS C:\Users\yello>
```

그림 **20_docker_stats** : docker stats : u2가 **CPU 0.00%, 메모리 약 3MB**만 쓰고 있어요. 아주 가벼워요.

6. 내 레시피 : Dockerfile과 이미지

Dockerfile은 도시락을 만드는 **레시피 카드**예요. 이미 있는 레시피(nginx)에 내 것을 조금 더해서 **내 이미지**를 만들었어요.

```
FROM nginx:alpine                          # 바탕 레시피: 웹 서버 nginx
LABEL org.opencontainers.image.title="forest-guide-web"   # 이름표
ENV APP_ENV=dev                            # 환경 구분 값
COPY site/ /usr/share/nginx/html/          # 내 숲해설 안내 페이지로 바꾸기
```

줄	하는 일	비유
FROM	바탕이 될 이미지를 골라요	기본 레시피 가져오기
LABEL	이미지에 이름표를 붙여요	도시락통에 이름 쓰기
ENV	환경 값을 정해요 (dev=개발용)	'오늘은 연습용' 표시
COPY	내 페이지를 웹 서버 자리에 넣어요	내 반찬 넣기

```
docker build -t forest-web:1.0 .      # 내 이미지 만들기 (맨 끝 점(.)은 '이 폴더에서'라는 뜻)
```

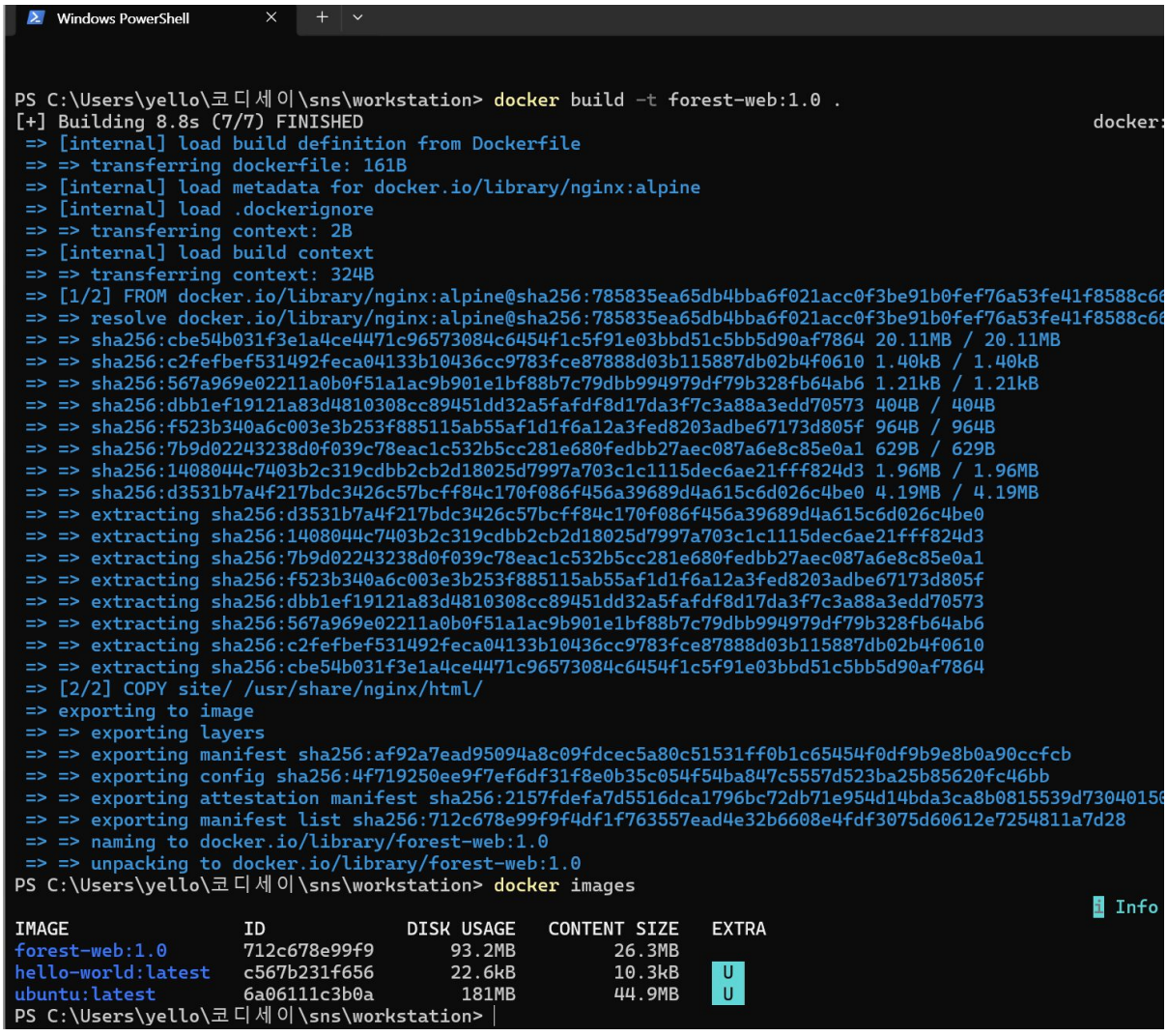


그림 21_docker_build : **내 이미지 forest-web:1.0**이 만들어졌어요. docker images에 93MB짜리로 보여요.

7. 정문 번호 : 포트 매핑

도시락 가게는 **문이 닫힌 방**이라, 밖에서 바로 들어올 수 없어요. 그래서 **정문 번호를 안쪽 방에 연결**해 줘야 해요. 그게 **포트 매핑**이에요.

```
docker run -d -p 8080:80 --name forest-8080 forest-web:1.0
docker run -d -p 8081:80 --name forest-8081 forest-web:1.0
#              ^내 PC 정문   ^도시락 안쪽 방
```

-p 8080:80 = "내 컴퓨터의 **8080번 문**으로 오면 도시락 안 **80번 방**으로 안내해 줘!"
정문 번호만 바꾸면(8080, 8081) **같은 레시피로 도시락을 여러 개** 만들 수 있어요.

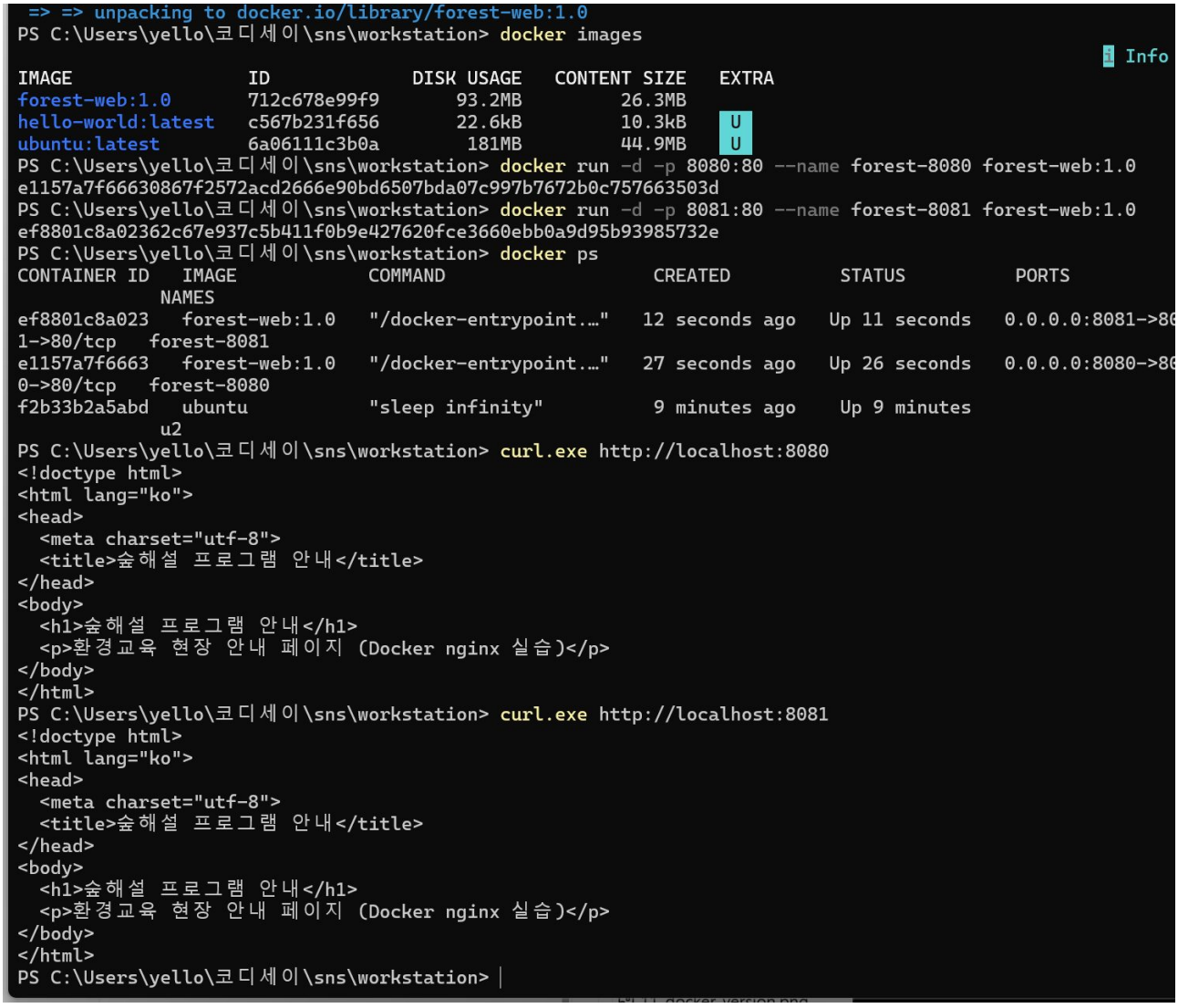


그림 22_port_mapping : 같은 이미지로 **컨테이너 2개**가 켜졌어요. PORTS에 8080->80, 8081->80이 보이고, curl로 두 곳 모두 숲해설 페이지가 나와요.

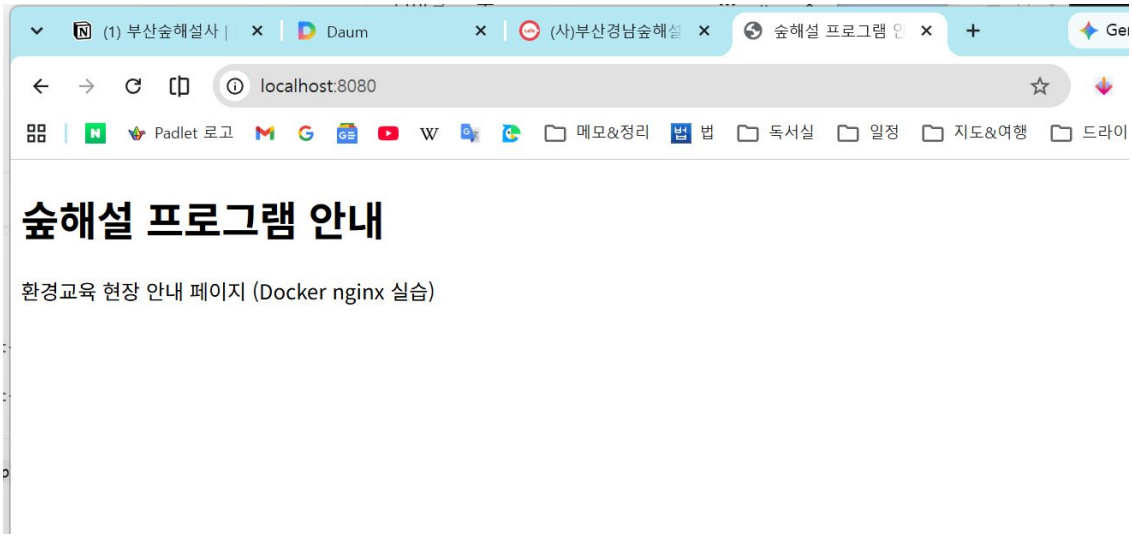
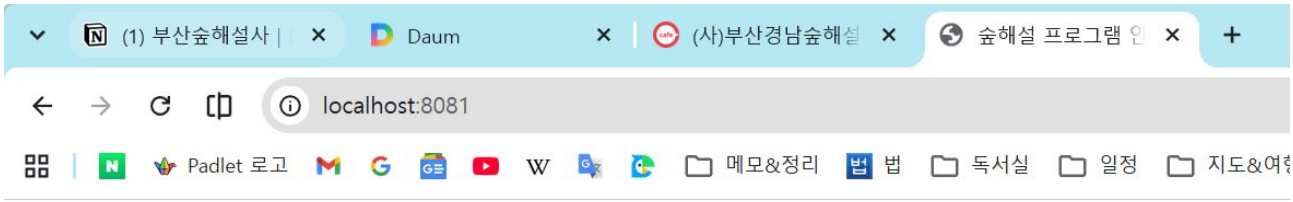


그림 23_browser_8080 : 브라우저 **주소창에 localhost:8080**이 보여요. 숲해설 프로그램 안내 페이지가 나와요.



숲해설 프로그램 안내

환경교육 현장 안내 페이지 (Docker nginx 실습)

그림 24_browser_8081 : **localhost:8081**도 같은 페이지예요. 정문만 다른 같은 도시락이에요.

가게 불이 꺼졌을 때

나중에 브라우저가 **연결 거부(ERR_CONNECTION_REFUSED)**를 보여 줬어요. 정문 앞에 갔는데 아무도 없는 거예요. 이유는 Docker 엔진이 꺼져서 도시락도 함께 멈췄기 때문이에요. 도시락은 **지워진 게 아니라 멈춰 있을 뿐**이라 다시 켰어요.

```
docker start forest-8080 forest-8081     # 멈춘 도시락 다시 켜기
```

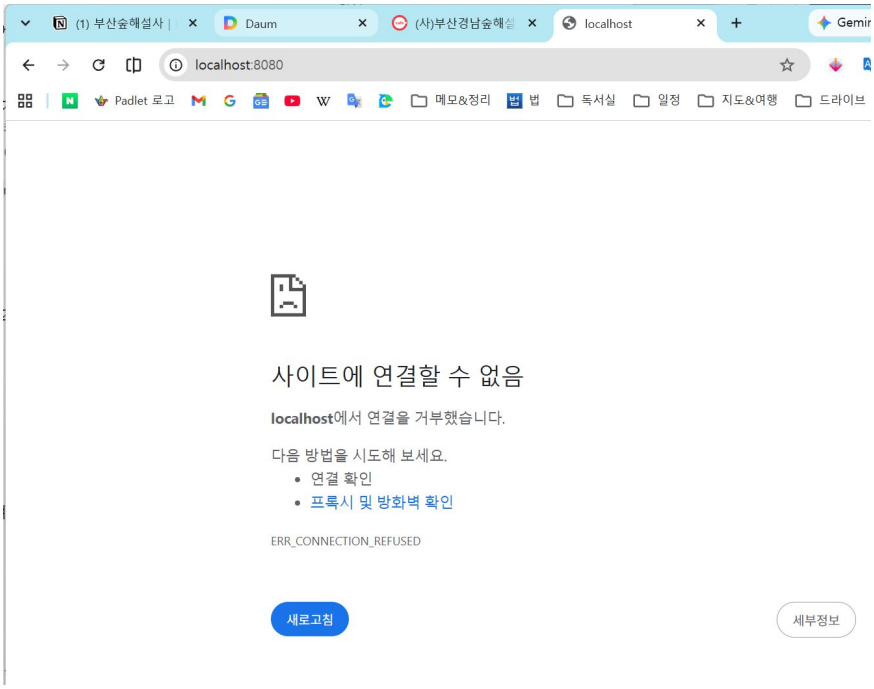


그림 25_connection_refused : 브라우저의 **연결 거부** 화면 (엔진이 꺼져 있을 때).

터미널

```
PS C:\Users\yello\코디세이\sns> docker start forest
-8080 forest-8081
forest-8080
forest-8081
PS C:\Users\yello\코디세이\sns> docker ps
CONTAINER ID   IMAGE            COMMAND
       CREATED        STATUS          PORTS
                              NAMES
ef8801c8a023   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 8 seconds   0.0.0.0:8081->8
0/tcp, [::]:8081->80/tcp   forest-8081
e1157a7f6663   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 9 seconds   0.0.0.0:8080->8
0/tcp, [::]:8080->80/tcp   forest-8080
PS C:\Users\yello\코디세이\sns> docker ps
CONTAINER ID   IMAGE            COMMAND
       CREATED        STATUS          PORTS
                              NAMES
ef8801c8a023   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 2 minutes   0.0.0.0:8081->8
0/tcp, [::]:8081->80/tcp   forest-8081
e1157a7f6663   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 2 minutes   0.0.0.0:8080->8
0/tcp, [::]:8080->80/tcp   forest-8080
PS C:\Users\yello\코디세이\sns> docker ps
CONTAINER ID   IMAGE            COMMAND
       CREATED        STATUS          PORTS
                              NAMES
ef8801c8a023   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 2 minutes   0.0.0.0:8081->8
0/tcp, [::]:8081->80/tcp   forest-8081
e1157a7f6663   forest-web:1.0   "/docker-entrypoint
.…"   21 hours ago   Up 2 minutes   0.0.0.0:8080->8
0/tcp, [::]:8080->80/tcp   forest-8080
PS C:\Users\yello\코디세이\sns>
```

그림 26_docker_start_ok : docker start 뒤 docker ps : 두 컨테이너가 **Up**이고 포트도 그대로예요.

8. 창문과 사물함 : 바인드 마운트, 볼륨

이름	쉬운 말	비유	언제 쓰나요?
바인드 마운트	내 컴퓨터 폴더와 도시락 안 폴더를 **한 창문**으로 이어요	같은 창문으로 같은 방 보기	고치면 **바로 바뀌어 보여야** 할 때(개발)
볼륨	Docker가 관리하는 **보관함**이에요	은행 사물함	도시락을 버려도 **남겨야** 하는 데이터

바인드 마운트 : 고치면 바로 바뀌어요

```
docker run -d -p 8082:80 --name bind-test -v "${PWD}₩site:/usr/share/nginx/html" nginx:alpine
# 내 PC의 site 폴더 <-> 도시락 안 /usr/share/nginx/html
```

내 컴퓨터에서 **index.html을 고친 뒤** 도시락을 다시 만들지 않았는데, 접속하면 **바뀐 내용**이 나왔어요.

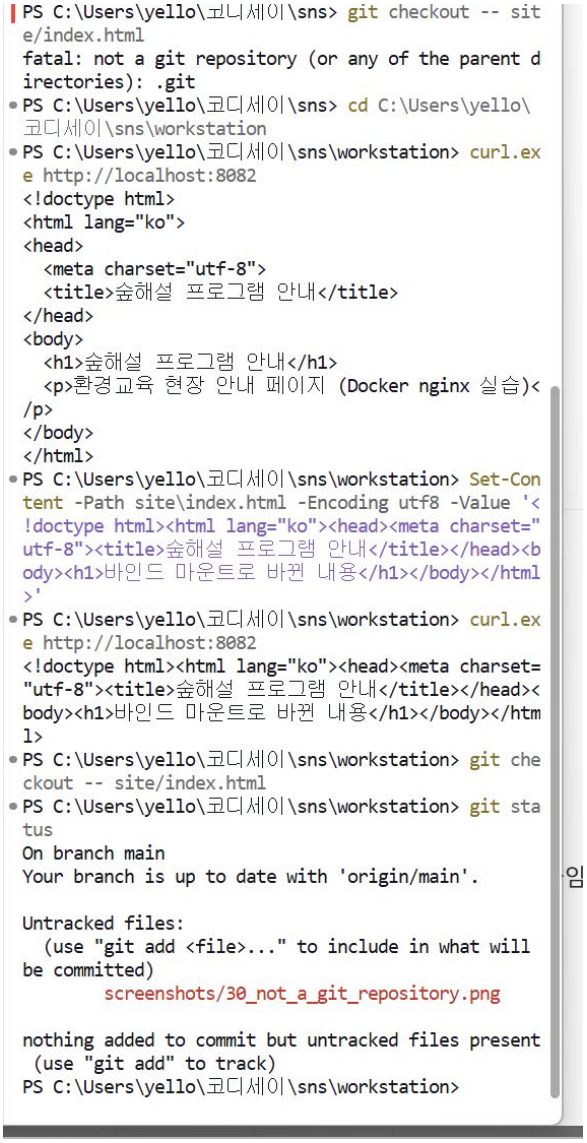


그림 27_bind_mount_before_after : **바뀌기 전**은 '숲해설 프로그램 안내', **고친 후**는 '바인드 마운트로 바뀐 내용'이 나와요. 도시락은 그대로인데 내용이 바뀌었어요.

볼륨 : 도시락을 버려도 남아요

1) 사물함(mydata)에 쪽지 **forest-note**를 넣고, 2) 도시락을 **버리고**, 3) 새 도시락에 같은 사물함을 연결하고, 4) 쪽지가 그대로인지 봤어요.

```
docker volume create mydata
docker run -d --name vol-test -v mydata:/data ubuntu sleep infinity
docker exec vol-test bash -c "echo forest-note > /data/hello.txt && cat /data/hello.txt"
docker rm -f vol-test                # 도시락 버리기!
docker run -d --name vol-test2 -v mydata:/data ubuntu sleep infinity
docker exec vol-test2 cat /data/hello.txt      # forest-note 가 그대로!
```

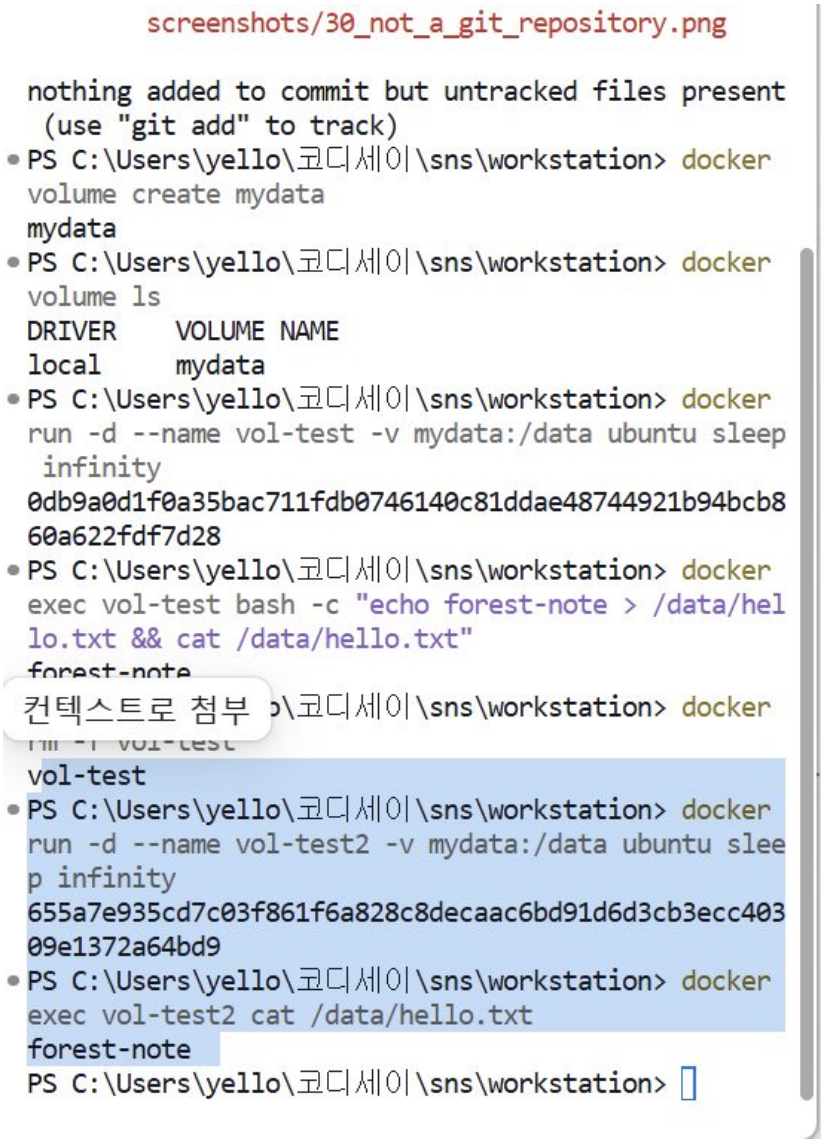


그림 32_volume_persist : 도시락(vol-test)을 지웠는데 새 도시락(vol-test2)에서 **forest-note**가 그대로 나왔어요. 데이터가 남는다는 증거예요.

9. 일기장과 클라우드 : Git과 GitHub

Git은 내 컴퓨터의 일기장이에요. 무엇을 바꿨는지 한 장씩 기록해요. **GitHub**는 그 일기장을 인터넷에 올려 두는 사물함이에요.

> **택배 보내기 순서** : **add**(상자에 담기) → **commit**(송장 붙여 저장) → **push**(택배 보내기)

```
git init                    # 새 일기장 만들기
git remote add origin ...   # 클라우드 사물함 주소 적어 두기
git add README.md           # 상자에 담기
git commit -m "설명"        # 저장하고 이름 붙이기
git push                    # GitHub로 보내기
```

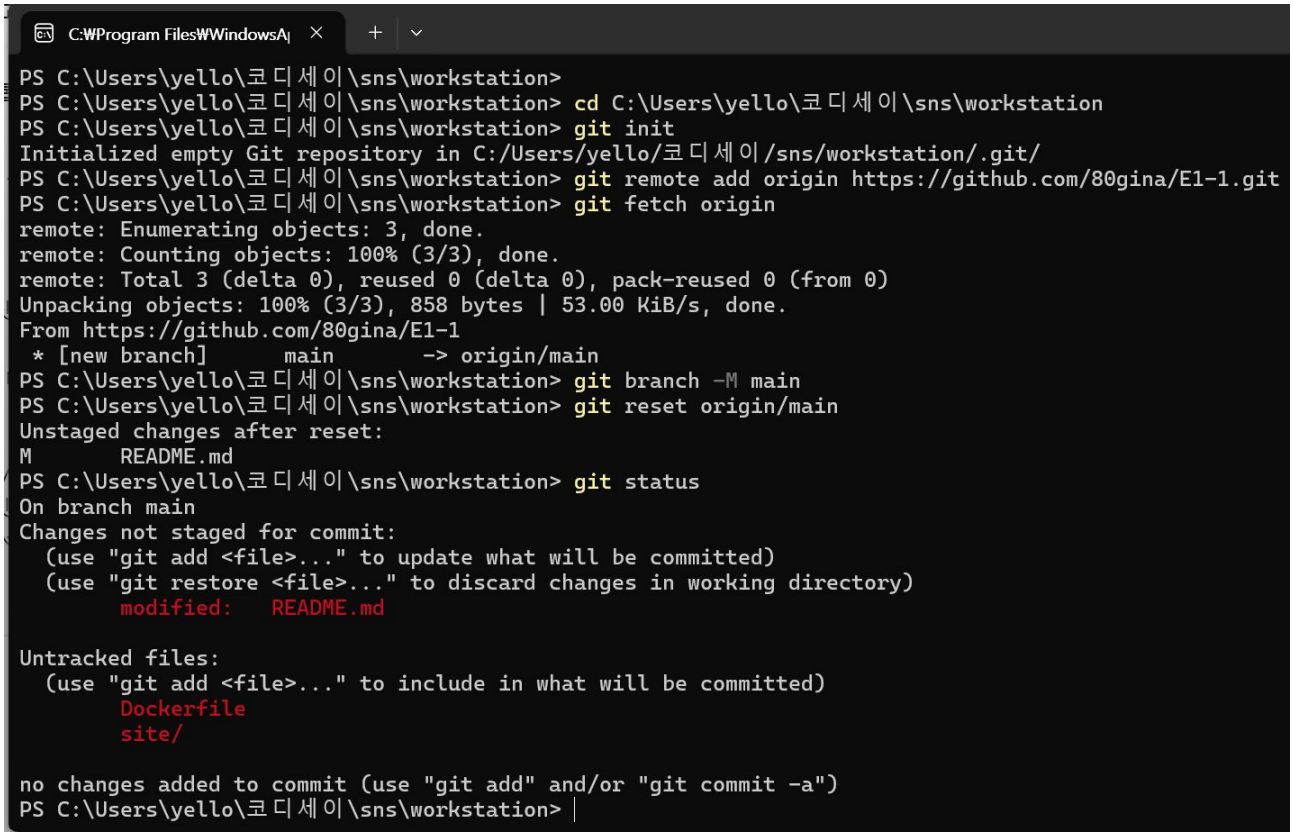


그림 02_git_remote_status : Git 일기장을 만들고(**git init**) GitHub 주소를 연결했어요(remote add). 아직 안 담은 파일이 빨간색으로 보여요.

```
PS C:\Users\yello\코디세이\sns\workstation> mkdir screenshots

    디렉터리: C:\Users\yello\코디세이\sns\workstation

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----        2026-11-02 오후 10:08                screenshots

PS C:\Users\yello\코디세이\sns\workstation> git add README.md practice screenshots
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\yello\코디세이\sns\workstation> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        new file:   practice/b.txt
        new file:   practice/sub/d.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Dockerfile
        site/

PS C:\Users\yello\코디세이\sns\workstation> git commit -m "Docs: 터미널 조작과 권한 실습 기록 추가"
[main 10437d0] Docs: 터미널 조작과 권한 실습 기록 추가
 3 files changed, 158 insertions(+), 2 deletions(-)
 create mode 100644 practice/b.txt
 create mode 100644 practice/sub/d.txt
PS C:\Users\yello\코디세이\sns\workstation>
```

그림 08a_git_add_commit : **add**로 담고 **commit**으로 저장했어요. 빨간 untracked 파일은 일부러 담지 않은 것들이에요.

```
PS C:\Users\yello\코디세이\sns\workstation> git commit -m "Docs: 터미널 조작과 권한 실습 기록 추가"
[main 10437d0] Docs: 터미널 조작과 권한 실습 기록 추가
 3 files changed, 158 insertions(+), 2 deletions(-)
 create mode 100644 practice/b.txt
 create mode 100644 practice/sub/d.txt
PS C:\Users\yello\코디세이\sns\workstation> git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 2.90 KiB | 741.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/80gina/E1-1.git
   b74028f..10437d0  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\yello\코디세이\sns\workstation>
```

그림 08_first_push : 처음으로 GitHub에 올렸어요(**push**). main -> main이 보이면 성공이에요.

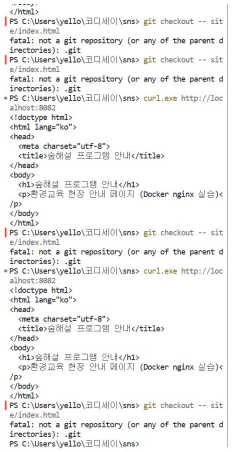


그림 30_not_a_git_repository : 엉뚱한 폴더(일기장이 없는 곳)에서 git을 쳤더니 **not a git repository**가 나왔어요. 폴더를 옮겨서 해결했어요.

Git 설정 확인 (개인정보는 가렸어요)

```
이\sns\workstation
PS C:\Users\yello\코디세이\sns\workstation> git config --global user.
name
Jina Kim
PS C:\Users\yello\코디세이\sns\workstation> git config --global init.
defaultBranch
main
PS C:\Users\yello\코디세이\sns\workstation> git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
core.autocrlf=true
core.quotepath=false
user.name=
user.email=
difftool.sourcetree.cmd=''
mergetool.sourcetree.cmd=''
mergetool.sourcetree.trustexitcode=true
init.defaultbranch=main
credential.https://huggingface.co.provider=generic
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/80gina/E1-1.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
branch.main.remote=origin
branch.main.merge=refs/heads/main
```

그림 **33_git_config** : **git config --list** : 기본 브랜치가 main이고, **이름과 이메일은 가렸어요**. 저장소 주소도 보여요.

```
init.defaultbranch=main
credential.https://huggingface.co.provider=generic
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/80gina/E1-1.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
branch.main.remote=origin
branch.main.merge=refs/heads/main
PS C:\Users\yello\코디세이\sns\workstation> git remote -v
origin  https://github.com/80gina/E1-1.git (fetch)
origin  https://github.com/80gina/E1-1.git (push)
PS C:\Users\yello\코디세이\sns\workstation>
```

그림 **34_git_remote** : **git remote -v** : origin이 GitHub의 E1-1 저장소를 가리켜요.

VSCode와 GitHub 연결

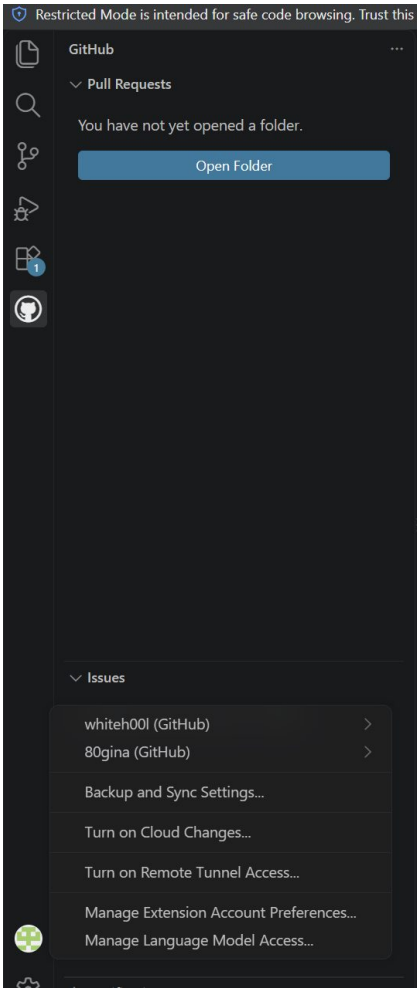


그림 35_vscode_github : VSCode 계정 메뉴에 **80gina (GitHub)**로 로그인된 모습이에요.

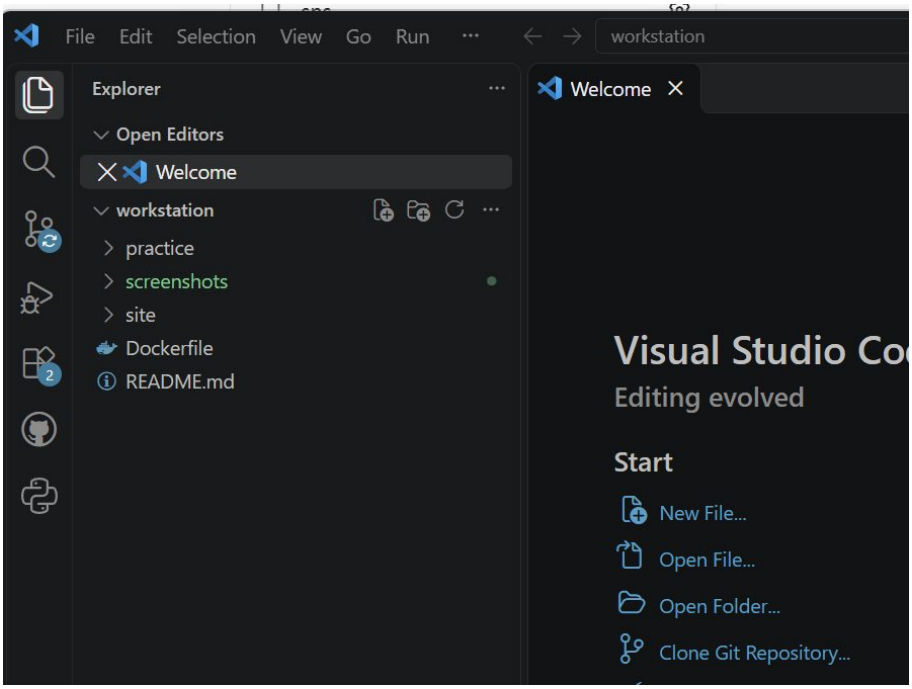


그림 36a_vscode_explorer : VSCode에서 **workstation 폴더**를 열었어요. 왼쪽에 파일 목록이 보여요.

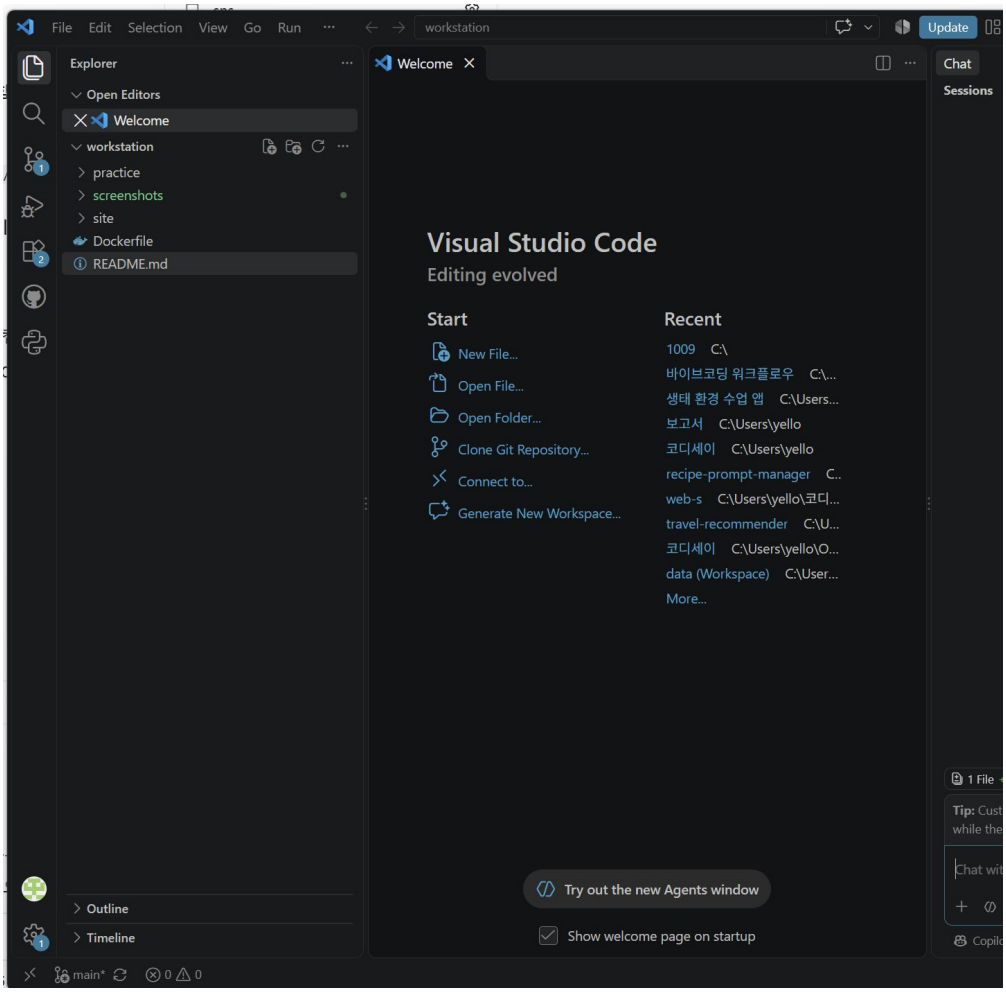


그림 36_vscode_source_control : VSCode 아래쪽에 **main** 브랜치가 보여요. 저장소와 잘 연결됐다는 뜻이에요.

10. 보너스 1~4 : Docker Compose와 환경 변수

Compose는 긴 docker run 명령을 **레시피 카드 한 장(docker-compose.yml)**에 적어 두는 거예요. 누구나 **docker compose up -d** 한 줄로 똑같이 만들 수 있어요.

보너스 1 : 한 개짜리 Compose

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8090:80"
    volumes:
      - ../../site:/usr/share/nginx/html:ro
```

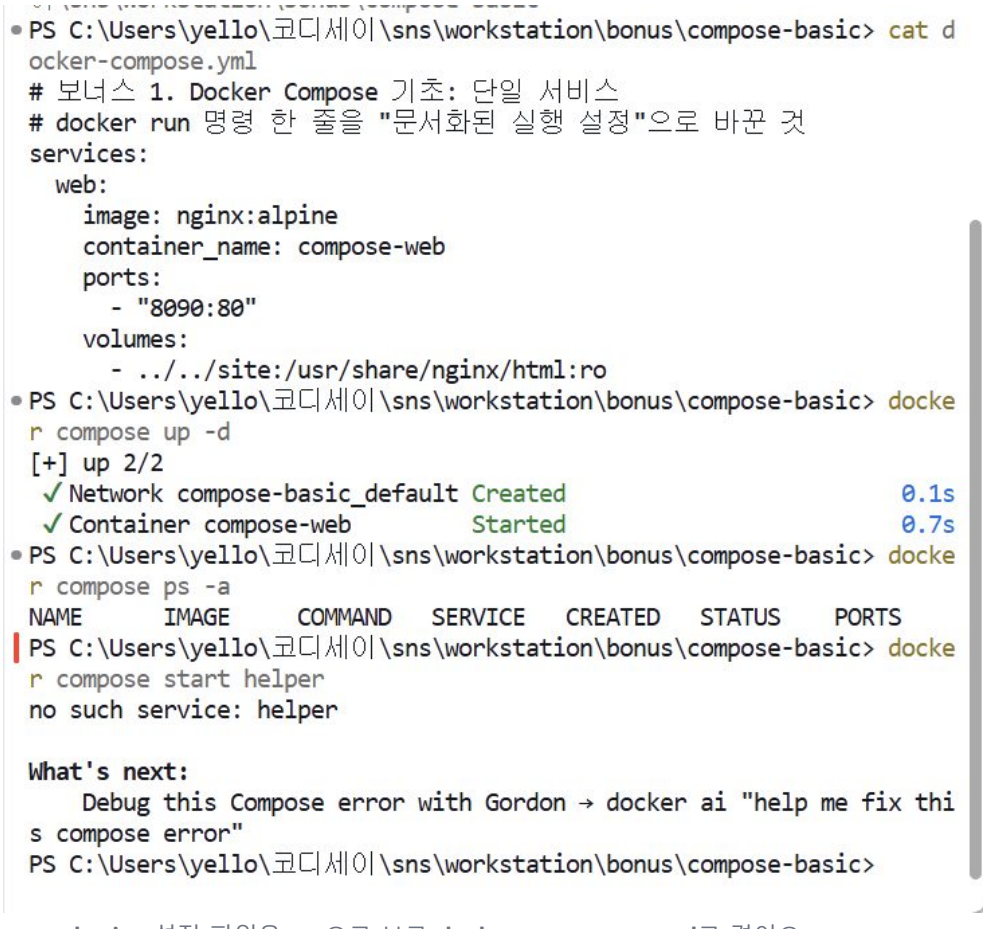


그림 40_compose_basic : 설정 파일을 **cat**으로 보고 **docker compose up -d**로 켰어요.

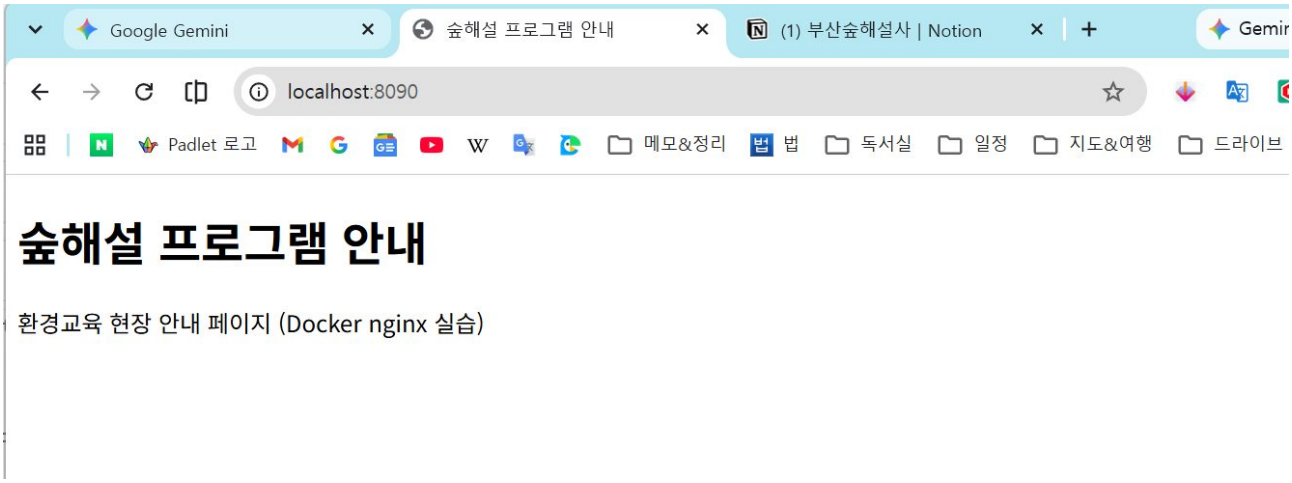


그림 41_compose_basic_browser : 브라우저 **localhost:8090** 접속 성공.

```
 create mode 100644 screenshots/36a_vscode_explorer.png
PS C:\Users\yello\코디세이\sns\workstation> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 226.38 KiB | 8.08 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects
To https://github.com/80gina/E1-1.git
   5d57517..a4d12f4  main -> main
PS C:\Users\yello\코디세이\sns\workstation> cd C:\Users\yello\코디세
이\sns\workstation
PS C:\Users\yello\코디세이\sns\workstation> git add README.md
warning: in the working copy of 'README.md', LF will be replaced by C
RLF the next time Git touches it
PS C:\Users\yello\코디세이\sns\workstation> git push
Everything up-to-date
PS C:\Users\yello\코디세이\sns\workstation> git push
Everything up-to-date
PS C:\Users\yello\코디세이\sns\workstation> docker compose ps
no configuration file provided: not found

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix thi
s compose error"
PS C:\Users\yello\코디세이\sns\workstation> curl.exe http://localhost
:8090
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>숲해설 프로그램 안내</title>
</head>
<body>
  <h1>숲해설 프로그램 안내</h1>
  <p>환경교육 현장 안내 페이지 (Docker nginx 실습)</p>
</body>
</html>
PS C:\Users\yello\코디세이\sns\workstation> docker compose down
no configuration file provided: not found

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix thi
s compose error"
PS C:\Users\yello\코디세이\sns\workstation> cd C:\Users\yello\코디세
이\sns\workstation\bonus\compose-basic
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic> docke
r compose ps
NAME          IMAGE          COMMAND                  SERVICE   CREA
TED          STATUS          PORTS
compose-web   nginx:alpine   "/docker-entrypoint.…"   web       58 se
conds ago   Up 57 seconds   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic>
```

그림 42_compose_no_config_fix : 처음엔 **설정 파일이 없는 폴더**에서 ps를 쳐서 'no configuration file'이 나왔고, **폴더를 옮기자** compose-web이 Up으로 보였어요.

```
이\sns\workstation\bonus\compose-basic
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic> docke
r compose ps
NAME          IMAGE          COMMAND                  SERVICE   CREA
TED          STATUS          PORTS
compose-web   nginx:alpine   "/docker-entrypoint.…"   web       58 se
conds ago   Up 57 seconds   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic> docke
r compose down
[+] down 2/2
 ✓ Container compose-web          Removed                          0.4s
 ✓ Network compose-basic_default  Removed                          0.3s
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic>
```

그림 43_compose_down : **docker compose down** : 도시락과 전용 길(네트워크)이 정리됐어요.

보너스 2 : 도시락 두 개가 서로 이름으로 부르기

web(내 페이지)과 **helper**(도우미)를 함께 켰어요. Compose는 둘을 **같은 길로 이어 주고** 서로를 **이름**으로 부를 수 있게 해 줘요.

```
exe http://localhost:8090
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>숲해설 프로그램 안내</title>
</head>
<body>
  <h1>숲해설 프로그램 안내</h1>
  <p>환경교육 현장 안내 페이지 (Docker nginx 실습)</p>
</body>
</html>
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic> docke
r compose down
[+] down 2/2
 ✓ Container compose-web          Removed                          3.5s
 ✓ Network compose-basic_default  Removed                          0.3s
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-basic> cd C:
\Users\yello\코디세이\sns\workstation\bonus\compose-multi
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> cat d
ocker-compose.yml
# 보너스 2·3·4. 멀티 컨테이너 + 운영 명령 + 환경 변수
# web(내 이미지)과 helper(보조 서비스) 2개를 함께 실행하고, 서비스 이
름으로 서로 통신한다.
services:
  web:
    image: forest-web:1.0          # 필수 과제에서 만든 커스텀 이미지
    container_name: compose-multi-web
    ports:
      - "${HOST_PORT:-8091}:80"    # 보너스 4: 환경 변수로 호스트 포
트 변경 (기본 8091)
    environment:
      - APP_ENV=${APP_ENV:-dev}    # 보너스 4: 환경 변수 주입 (기본 d
ev)

  helper:
    image: alpine:3.20
    container_name: compose-multi-helper
    command: sleep infinity        # 계속 켜 두고 exec로 web에 요청을
 보내 본다
    environment:
      - APP_ENV=${APP_ENV:-dev}
    depends_on:
      - web
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose up -d
[+] up 7/7
 ✓ Image alpine:3.20              Pulled                           4.5s
 ✓ Network compose-multi_default  Created                          0.0s
 ✓ Container compose-multi-web    Started                          0.6s
 ✓ Container compose-multi-helper Started                          0.7s
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose ps
NAME                  IMAGE            COMMAND                  SER
VICE    CREATED         STATUS         PORTS
compose-multi-helper   alpine:3.20      "sleep infinity"         hel
per     8 seconds ago   Up 7 seconds
compose-multi-web      forest-web:1.0   "/docker-entrypoint.…"   web
       9 seconds ago   Up 8 seconds   0.0.0.0:8091->80/tcp, [::]:8091-
>80/tcp
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi>
```

그림 44_compose_multi_up : docker compose up -d 로 **web과 helper가 함께 Started** 됐어요. 둘 다 Up이에요.

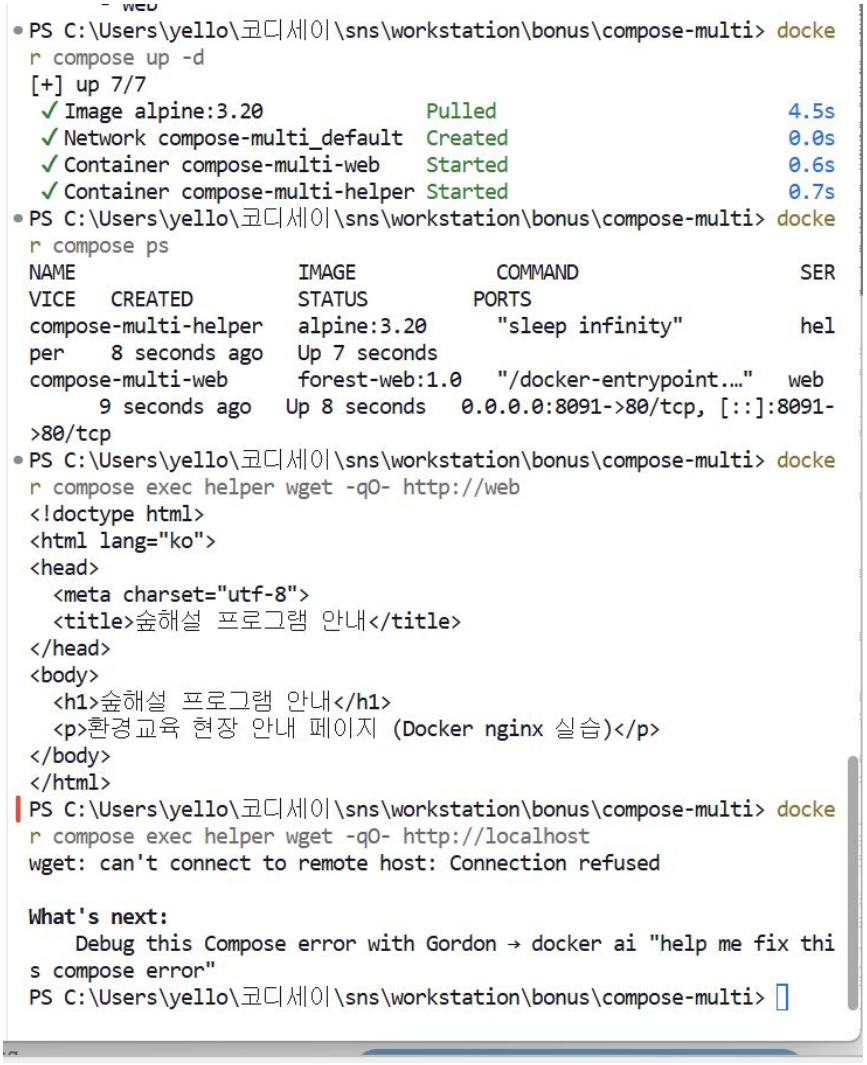


그림 45_compose_multi_network : helper가 **http://web**으로 부르면 성공(HTML이 와요), **http://localhost**로 부르면 거부(Connection refused)예요. 도시락 안의 localhost는 **자기 자신**이라서요.

보너스 3 : 가게 상태 확인 루틴 (up, ps, logs, down)

```
docker compose up -d       # 켜기
docker compose ps          # 출석 확인
docker compose logs web    # 기록 보기
docker compose down        # 끄기
```

```
</head>
<body>
  <h1>숲해설 프로그램 안내</h1>
  <p>환경교육 현장 안내 페이지 (Docker nginx 실습)</p>
</body>
</html>
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose logs web --tail 5
compose-multi-web  | 2026/11/03 11:05:04 [notice] 1#1: start worker p
rocess 40
compose-multi-web  | 2026/11/03 11:05:04 [notice] 1#1: start worker p
rocess 41
compose-multi-web  | 172.18.0.3 - - [03/Nov/2026:11:06:02 +0000] "GET
 / HTTP/1.1" 200 252 "-" "Wget" "-"
compose-multi-web  | 172.18.0.3 - - [03/Nov/2026:11:06:27 +0000] "GET
 / HTTP/1.1" 200 252 "-" "Wget" "-"
compose-multi-web  | 172.18.0.1 - - [03/Nov/2026:11:08:01 +0000] "GET
 / HTTP/1.1" 200 252 "-" "curl/8.22.0" "-"
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose ps
NAME                    IMAGE             COMMAND                  SER
VICE    CREATED          STATUS          PORTS
compose-multi-helper    alpine:3.20       "sleep infinity"         hel
per     3 minutes ago    Up 3 minutes
compose-multi-web       forest-web:1.0    "/docker-entrypoint.…"   web
       3 minutes ago    Up 3 minutes    0.0.0.0:8091->80/tcp, [::]:8091-
>80/tcp
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi>
```

그림 47_compose_ops_logs : **logs** : web이 받은 요청 기록이 나와요. 누가 요청했는지(Wget, curl)까지 알 수 있어요.

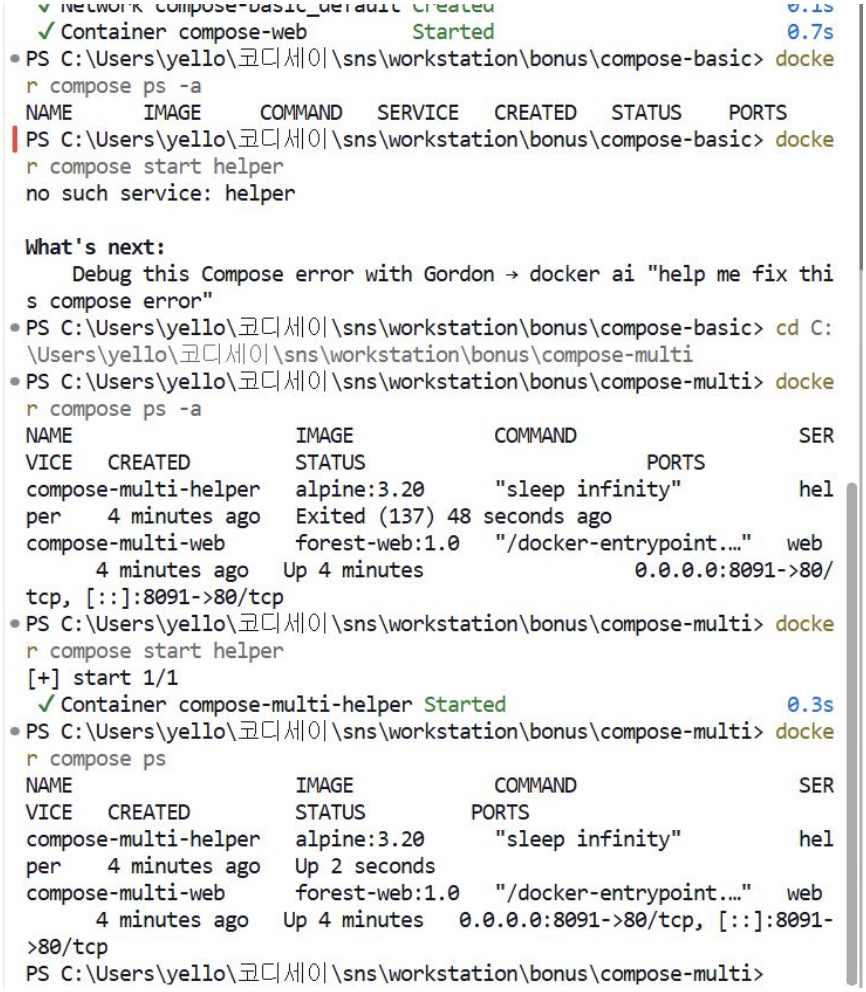


그림 **48_compose_ops_stop_start** : **helper만 멈췄다(Exited) 다시 켜기(Up)**. web은 계속 켜져 있었어요.

보너스 4 : 레시피는 그대로, 옵션표만 바꾸기

환경 변수로 포트(8091→8095)와 모드(dev→prod)를 바꿨어요. **파일을 한 글자도 고치지 않았어요.** 이렇게 '설정'과 '레시피'를 나누는 걸 **설정과 코드의 분리**라고 해요.

```
$env:HOST_PORT="8095"       # 이 창에서만 쓰는 임시 값
$env:APP_ENV="prod"
docker compose up -d
```

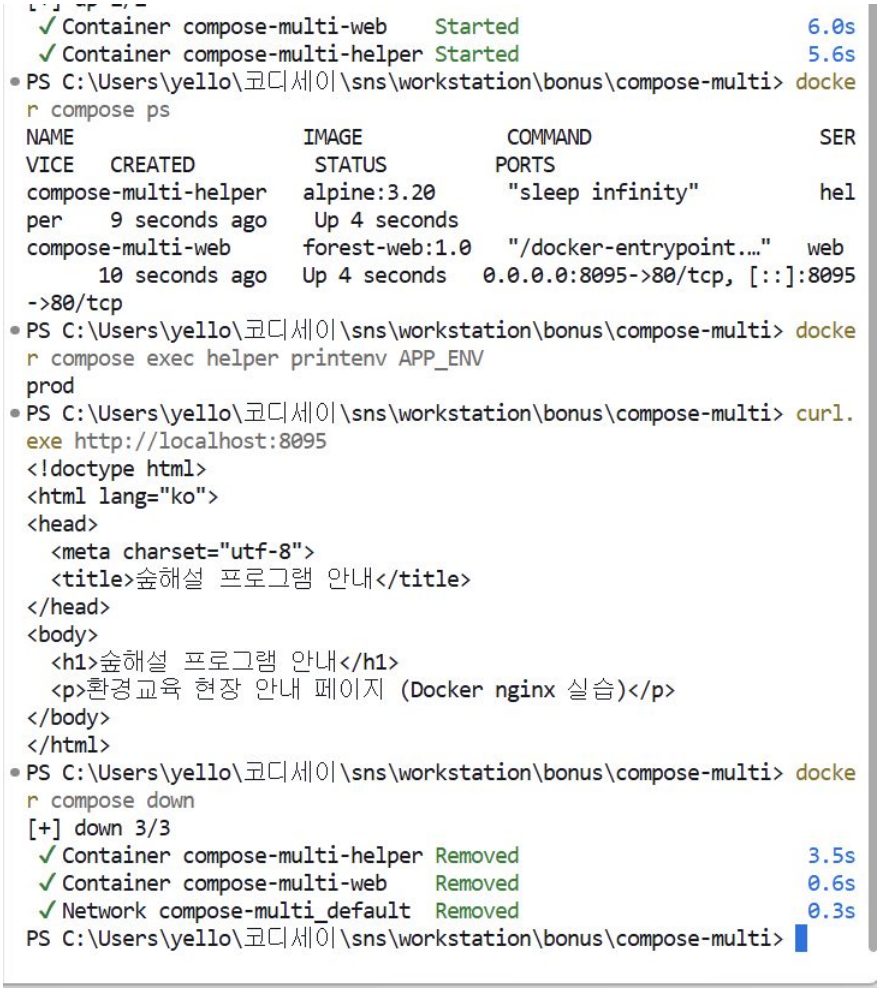


그림 49_compose_env_override : 바꾼 값으로 켜서 확인한 뒤 down까지 한 전체 흐름이에요. 포트가 **8095**, 모드가 **prod**예요.

```
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose ps
NAME                    IMAGE             COMMAND                  SER
VICE    CREATED          STATUS          PORTS
compose-multi-helper    alpine:3.20       "sleep infinity"         hel
per     9 seconds ago    Up 4 seconds
compose-multi-web       forest-web:1.0    "/docker-entrypoint.…"   web
       10 seconds ago   Up 4 seconds   0.0.0.0:8095->80/tcp, [::]:8095
->80/tcp
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose exec helper printenv APP_ENV
prod
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> curl.
exe http://localhost:8095
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <title>숲해설 프로그램 안내</title>
</head>
<body>
  <h1>숲해설 프로그램 안내</h1>
  <p>환경교육 현장 안내 페이지 (Docker nginx 실습)</p>
</body>
</html>
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi>
```

그림 49a_compose_env_check : ps에 8095->80, printenv APP_ENV가 prod, curl로 8095 접속 성공.

```
 ✓ Container compose-multi-web      Removed                    0.6s
 ✓ Network compose-multi_default    Removed                    0.3s
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> docke
r compose down
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> Remov
e-Item Env:HOST_PORT
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi> Remov
e-Item Env:APP_ENV
PS C:\Users\yello\코디세이\sns\workstation\bonus\compose-multi>
```

그림 50_compose_env_cleanup : 임시 값 지우기(Remove-Item Env:…)까지 **오류 없이** 끝났어요.

11. 보너스 5 : SSH 열쇠와 자물쇠

지금까지는 GitHub에 올릴 때 **신분증(로그인)**을 보여 줬어요. 이번엔 **열쇠와 자물쇠**를 써 봤어요.

이름	비유	어디에 두나요?	남에게 보여 줘도 되나요?
공개키 (.pub)	자물쇠	GitHub 사물함에 달아 둬요	**네**, 괜찮아요
개인키	열쇠	**내 컴퓨터 안에만**	**절대 안 돼요!**

> **지킨 약속** : 열쇠(개인키)는 화면에도, 문서에도, 저장소에도 **올리지 않았어요.** 공개키와 지문만 보여요.

```
ssh-keygen -t ed25519 -C "github-ssh-e1"    # 열쇠와 자물쇠 한 쌍 만들기
ssh -T git@github.com                       # 열쇠로 열리는지 시험
git remote set-url origin git@github.com:80gina/E1-1.git   # 올리는 길을 SSH로 바꾸기
```

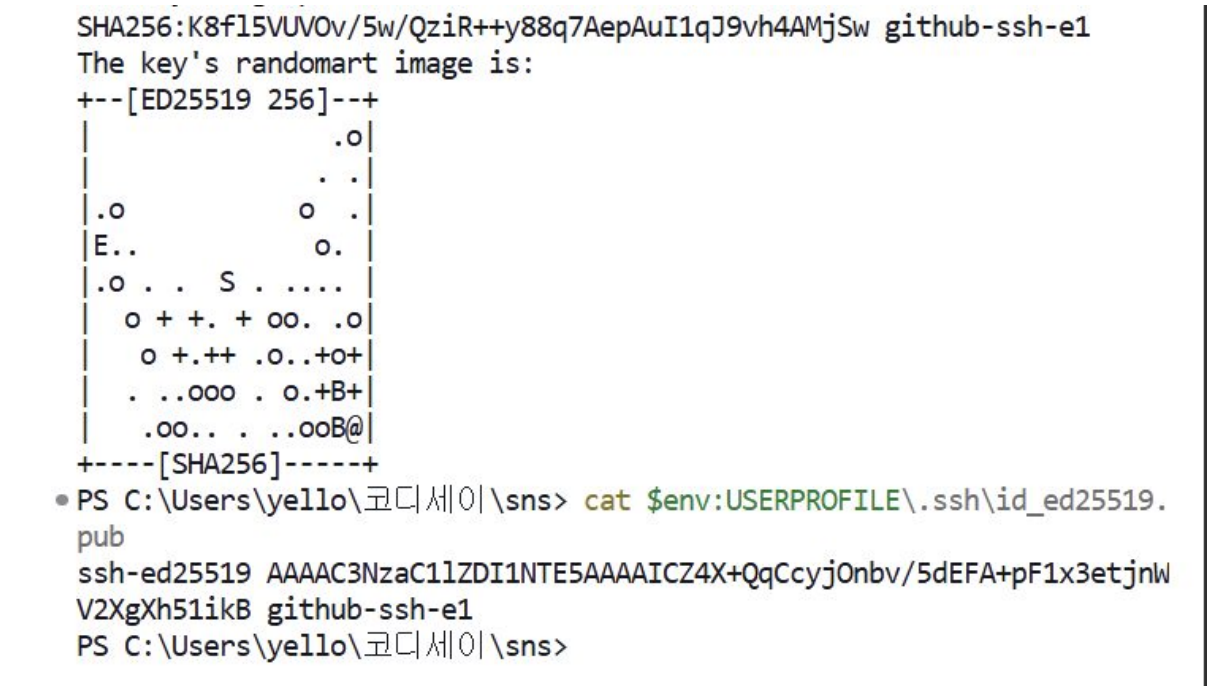


그림 51_ssh_keygen : 열쇠 한 쌍을 만들고 **공개키(.pub)** 한 줄과 지문을 확인했어요.



그림 52_github_ssh_key_added : GitHub에 자물쇠를 달았어요. 이름 **내 윈도우 PC**와 **지문**이 보여요. 터미널에서 본 지문과 같아요.

```
Hi 80gina! You've successfully authenticated, but GitHub does not pro
vide shell access.
PS C:\Users\yello\코디세이\sns>
```

그림 53_ssh_test : **Hi 80gina!** 열쇠로 사물함이 열렸어요.

```
PS C:\Users\yello\코디세이\sns> cd C:\Users\yello\코디세이\sns\workst
ation
PS C:\Users\yello\코디세이\sns\workstation> git remote set-url origin
 git@github.com:80gina/E1-1.git
PS C:\Users\yello\코디세이\sns\workstation> git remote -v
origin  git@github.com:80gina/E1-1.git (fetch)
origin  git@github.com:80gina/E1-1.git (push)
PS C:\Users\yello\코디세이\sns\workstation>
```

그림 54_ssh_remote : 올리는 주소가 **git@github.com:80gina/E1-1.git**(SSH)로 바뀌었어요.

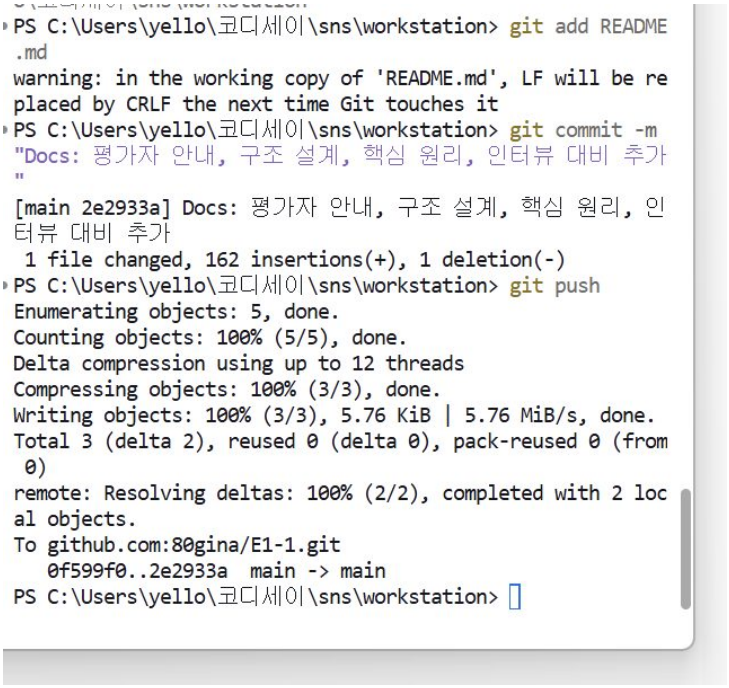


그림 55_ssh_push : **To github.com:80gina/E1-1.git** : SSH로 push에 성공했어요. 비밀번호 없이요.

12. 포트가 겹치면? 직접 일으켜 본 실험

같은 정문 번호(8080)를 **두 도시락에** 연결하려고 하면 어떻게 될까요? 일부러 해 봤어요. **한 번호는 한 곳에만** 쓸 수 있어요.

```
docker run -d -p 8080:80 --name conflict-a nginx:alpine   # 8080 차지
docker run -d -p 8080:80 --name conflict-b nginx:alpine   # 같은 번호! -> 실패
```

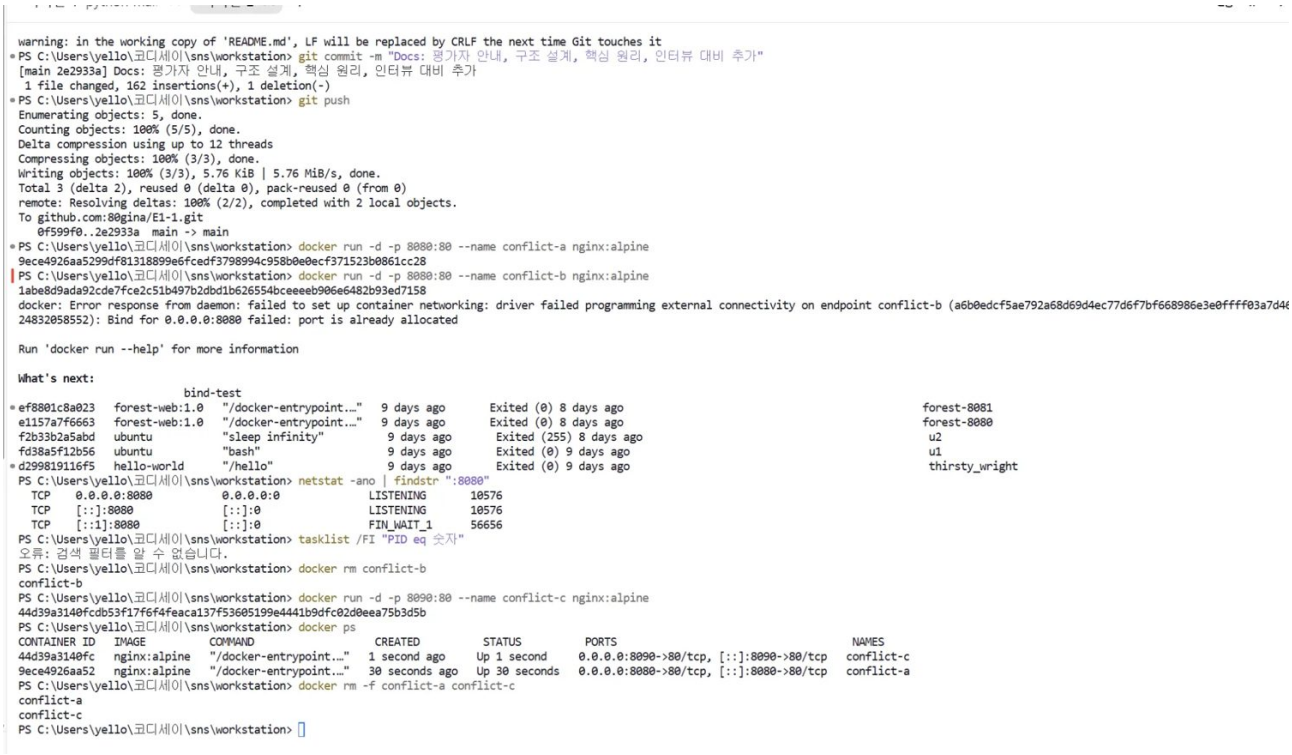


그림 **56_port_conflict_error** : 두 번째에서 **Bind for 0.0.0.0:8080 failed: port is already allocated** (이미 사용 중이에요)라는 오류가 나왔어요. 일부러 일으킨 오류라 정상이에요.

누가 쓰고 있을까? 조사하는 순서

순서	할 일	명령
1	오류 메시지에서 **어떤 번호**인지 읽어요	8080
2	Docker 안에서 쓰는지 봐요	docker ps (PORTS 칸)
3	컴퓨터 전체에서 쓰는지 봐요	netstat -ano | findstr ":8080"
4	그 번호(PID)가 어떤 프로그램인지 봐요	tasklist /FI "PID eq 10576"
5	고쳐요: 쓰는 쪽을 멈추거나 **내 번호를 바꿔요**	-p 8090:80

```
What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
PS C:\Users\yello\코디세이\sns\workstation> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          POR
TS
               NAMES
9ece4926aa52   nginx:alpine   "/docker-entrypoint.…"   18 seconds ago   Up 18 seconds   0.0.
0.0:8080->80/tcp, [::]:8080->80/tcp   conflict-a
PS C:\Users\yello\코디세이\sns\workstation> docker ps -a
CONTAINER ID   IMAGE            COMMAND
CREATED          STATUS                     PORTS
                         NAMES
1abe8d9ada92   nginx:alpine     "/docker-entrypoint.…"   12 seconds ago   Creat
                        bind-test
ef8801c8a023   forest-web:1.0   "/docker-entrypoint.…"   9 days ago       Exited (0) 8 days
ago                                              forest-8081
e1157a7f6663   forest-web:1.0   "/docker-entrypoint.…"   9 days ago       Exited (0) 8 days
ago                                              forest-8080
f2b33b2a5abd   ubuntu           "sleep infinity"          9 days ago       Exited (255) 8 da
ys ago                                           u2
fd38a5f12b56   ubuntu           "bash"                    9 days ago       Exited (0) 9 days
 ago                                             u1
d299819116f5   hello-world      "/hello"                  9 days ago       Exited (0) 9 days
 ago                                             thirsty_wright
PS C:\Users\yello\코디세이\sns\workstation> netstat -ano | findstr ":8080"
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING       10576
  TCP    [::]:8080              [::]:0                 LISTENING       10576
  TCP    [::1]:8080             [::]:0                 FIN_WAIT_1      56656
PS C:\Users\yello\코디세이\sns\workstation> tasklist /FI "PID eq 숫자"
오류: 검색 필터를 알 수 없습니다.
PS C:\Users\yello\코디세이\sns\workstation> docker rm conflict-b
conflict-b
PS C:\Users\yello\코디세이\sns\workstation> docker run -d -p 8090:80 --name conflict-c nginx
:alpine
44d39a3140fcdb53f17f6f4feaca137f53605199e4441b9dfc02d0eea75b3d5b
PS C:\Users\yello\코디세이\sns\workstation> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          POR
TS                                      NAMES
44d39a3140fc   nginx:alpine   "/docker-entrypoint.…"   1 second ago     Up 1 second     0.0.
0.0:8090->80/tcp, [::]:8090->80/tcp   conflict-c
9ece4926aa52   nginx:alpine   "/docker-entrypoint.…"   30 seconds ago   Up 30 seconds   0.0.
0.0:8080->80/tcp, [::]:8080->80/tcp   conflict-a
PS C:\Users\yello\코디세이\sns\workstation> docker rm -f conflict-a conflict-c
conflict-a
conflict-c
PS C:\Users\yello\코디세이\sns\workstation>
```

그림 57_port_conflict_diagnosis : 조사와 해결의 전체 흐름이에요. **LISTENING 10576**이 8080을 잡고 있고, 번호를 **8090**으로 바꾸자 두 도시락이 동시에 Up이에요.

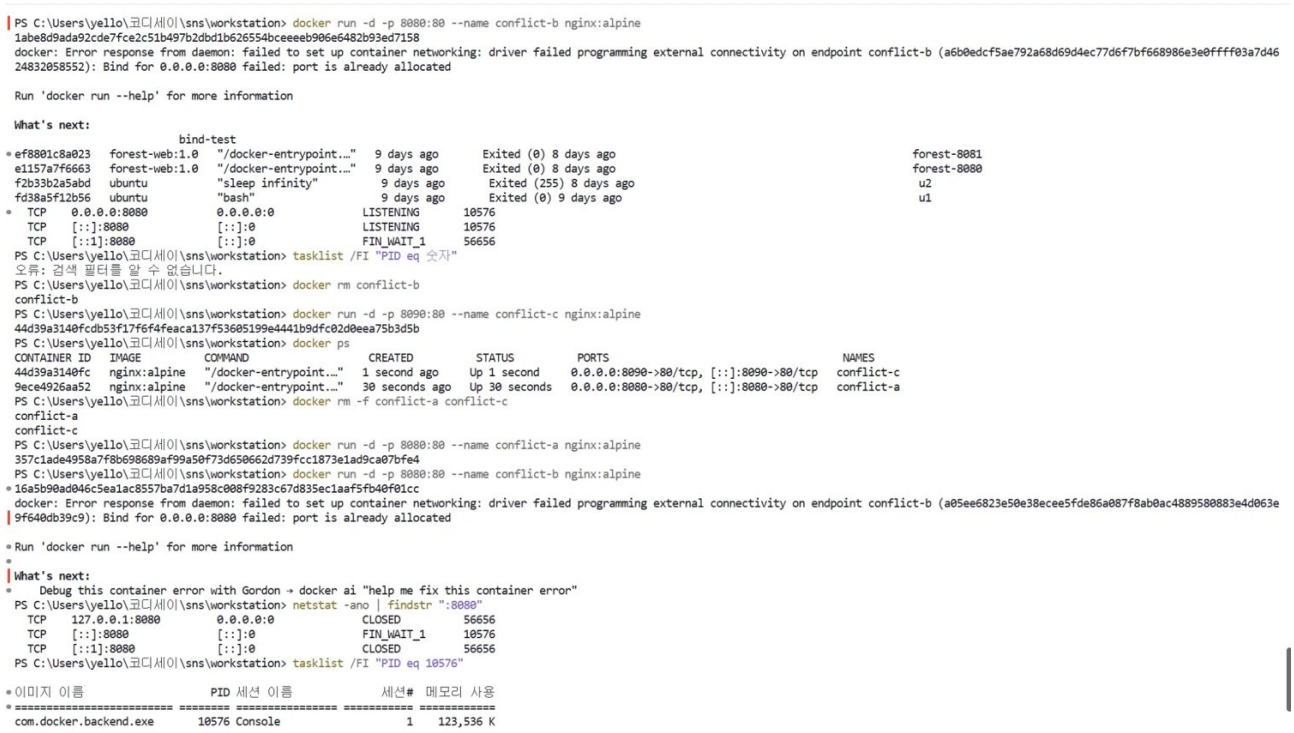


그림 **57a_port_conflict_pid** : PID **10576**은 **com.docker.backend.exe** (Docker 자신)였어요.

새로 알게 된 것 : 실패한 도시락(conflict-b)은 **Created 상태로 남아요**. 그래서 **docker rm**으로 정리해야 해요. 또, 처음에 명령에 **'숫자'라는 글자**를 그대로 넣었다가 오류가 났어요. 진짜 번호(10576)로 바꿔 넣어 해결했어요.

13. 어려웠던 일과 해결 방법

오류는 **고장이 아니라 "여기를 확인해 봐!"라는 안내**예요. 만난 오류와 푼 방법이에요.

오류	쉬운 말	해결
failed to connect to the docker API	도시락 가게의 **불이 꺼져** 있어요	Docker Desktop을 켰어요 (그림 10, 11a)
WSL not installed	도시락을 데울 **전자레인지가 없어요**	wsl --install 후 재부팅 (그림 12)
ERR_CONNECTION_REFUSED	정문 앞에 갔는데 **아무도 없어요**	docker start로 다시 켰어요 (그림 25, 26)
docker: command not found (도시락 안)	안에는 **docker가 없어요**	exit로 나와서 바깥에서 입력 (그림 16a)
not a git repository	**엉뚱한 서랍**을 열었어요	맞는 폴더로 cd (그림 30)
no configuration file provided / no such service	**다른 카드 상자**에서 레시피를 찾았어요	compose 파일이 있는 폴더로 이동 (그림 42)
port is already allocated (일부러)	같은 정문 번호를 **두 번** 썼어요	번호를 바꿨어요 (그림 56, 57)

가장 자주 한 실수는 **"지금 내가 어느 폴더에 있는지 안 보고 명령을 친 것"**이에요. 그래서 명령을 치기 전에 **화면 맨 끝의 폴더 이름**을 먼저 보는 습관을 들였어요.

14. 질문과 쉬운 대답

Q1. 이미지와 컨테이너는 뭐가 달라요?

	이미지	컨테이너
만들 때	docker build로 **만들어요** (레시피)	이미지로 docker run해서 **생겨요** (도시락)
움직일 때	움직이지 않아요. 읽기만 돼요.	실제로 **움직여요**
바꿀 때	고치려면 다시 build해서 **새 이미지**	안에서 바꾼 건 그 도시락에만. 버리면 사라져요

레시피 하나(forest-web:1.0)로 도시락 두 개(forest-8080, forest-8081)를 만들었어요. (그림 22~24)

Q2. 도시락 안 번호로 왜 바로 못 들어가요? 포트 매핑은 왜 필요해요?

도시락은 **자기만의 닫힌 방**에서 움직여요. 안에서 80번 문을 열어도 내 컴퓨터의 80번 문과는 다른 방이에요. 그래서 **-p 내번호:안쪽번호**로 연결해 줘야 브라우저가 들어갈 수 있어요. 내 번호만 바꾸면 같은 도시락을 여러 개 켤 수 있어요.

Q3. 절대 경로와 상대 경로는 언제 써요?

	모양	비유	이럴 때
절대 경로	C:₩Users₩yello₩...	'부산시 ○○구 ○○동 3번지'	어디서 시작하든 **똑같은 곳**을 가리켜야 할 때
상대 경로	site₩index.html	'우리 집에서 옆 건물'	프로젝트 안에서 **어디에 옮겨 놔도** 동작해야 할 때

Q4. 755, 644는 어떻게 읽어요?

읽기=4, 쓰기=2, 실행=1을 더해서 **주인 / 모둠 / 다른 사람** 순서로 써요. **755** = 7(전부) 5(읽기+실행) 5(읽기+실행), **644** = 6(읽기+쓰기) 4(읽기) 4(읽기)예요. (그림 17)

Q5. 포트가 이미 쓰이고 있으면 어떤 순서로 알아봐요?

① 메시지에서 번호 읽기 → ② docker ps로 Docker 안 확인 → ③ netstat로 컴퓨터 전체 확인 → ④ tasklist로 PID가 누군지 확인 → ⑤ 쓰는 쪽을 멈추거나 내 번호를 바꾸기. 직접 해 봤어요. (그림 56, 57, 57a)

Q6. 컨테이너를 지웠더니 데이터가 사라지면 어떻게 막아요?

- **볼륨** : 도시락과 따로 보관해요. (그림 32)
- **바인드 마운트** : 내 컴퓨터 폴더에 직접 두어요. (그림 27)
- **백업** : 볼륨 내용을 파일로 따로 저장해 둬요.

주의: **docker compose down -v**나 **docker volume rm**은 사물함까지 지워요.

Q7. 가장 어려웠던 일은 뭐였고, 어떻게 풀었어요?

단계	내용
증상	브라우저가 localhost:8080 연결을 거부했어요. (그림 25)

단계	내용
가설(추측)	① 도시락이 꺼졌나? ② 포트 연결이 틀렸나? ③ 엔진이 꺼졌나?
확인	docker ps -a에서 Exited (255), docker info도 연결 실패 → ①과 ③이 맞고 ②는 아니에요.
조치	Docker Desktop을 켜고 docker start로 다시 켰어요. (그림 26)
결과	접속이 다시 됐어요. (그림 23) 엔진이 꺼져도 도시락은 지워지지 않고 멈춰 있을 뿐이에요.

Q8. Git과 GitHub는 뭐가 달라요?

Git은 내 컴퓨터에서 바뀐 내용을 기록하는 도구(일기장)이고, **GitHub**는 그 기록을 인터넷에 올려 공유하고 같이 일하는 곳(클라우드 사물함)이에요.

15. 평가 문항 찾아보기

평가 항목	확인할 것	이 문서의 장	증거 그림
1. 기능 동작	폴더·파일 만들기, 옮기기, 지우기	2장	01, 03, 04
	권한 변경 결과	3장	05, 06, 07, 17
	docker --version, 동작 상태	4장	11_docker_version, 11_docker_info_ok
	hello-world 실행	5장	14
	이미지/컨테이너 목록, 정리	5장, 8장, 10장	15, 18, 18a, 20, 32, 43
	Dockerfile로 빌드	6장	21
	매핑된 포트로 접속	7장	22, 23, 24
	볼륨 데이터가 삭제 후에도 유지	8장	32
	Git 설정과 GitHub 연동	9장	33~36, 53~55
2. 구조 설계	폴더 구성, 포트·볼륨 재현	README 6-2장	-
3. 핵심 원리	이미지 vs 컨테이너, 포트, 경로, 권한	14장 Q1~Q4	22~24, 27, 45, 17
4. 심층 인터뷰	포트 충돌 진단, 데이터 유실 대안, 어려웠던 점	12장, 14장 Q5~Q7	25, 26, 32, 56, 57, 57a
5. 보너스	Compose, 환경 변수, SSH	10장, 11장	40~55

제출 저장소 : **https://github.com/80gina/E1-1** (브랜치 **main**). 증거 사진은 저장소의 **screenshots** 폴더에 번호순으로 들어 있어요.

부록. 증거 사진 목록

번호/이름	쓰인 장
01_pwd_ls.png	2. 터미널 : 글자로 시키는 리모컨
02_git_remote_status.png	9. 일기장과 클라우드 : Git과 GitHub
03_create_files.png	2. 터미널 : 글자로 시키는 리모컨
04_copy_move_delete.png	2. 터미널 : 글자로 시키는 리모컨
05_file_permission.png	3. 권한 : 서랍 열쇠 규칙
06_dir_permission.png	3. 권한 : 서랍 열쇠 규칙
07_dir_restore.png	3. 권한 : 서랍 열쇠 규칙
08_first_push.png	9. 일기장과 클라우드 : Git과 GitHub
08a_git_add_commit.png	9. 일기장과 클라우드 : Git과 GitHub
09a_docker_before.png	4. Docker 준비 : 도시락 가게 문 열기
10_docker_engine_off.png	4. Docker 준비 : 도시락 가게 문 열기
11_docker_info_ok.png	4. Docker 준비 : 도시락 가게 문 열기
11_docker_version.png	4. Docker 준비 : 도시락 가게 문 열기
11a_docker_desktop_running.png	4. Docker 준비 : 도시락 가게 문 열기
12_wsl_not_installed.png	4. Docker 준비 : 도시락 가게 문 열기
14_hello_world.png	5. 첫 도시락 : hello-world와 ubuntu
15_images_ps.png	5. 첫 도시락 : hello-world와 ubuntu
16a_docker_inside_container_error.png	5. 첫 도시락 : hello-world와 ubuntu
17_chmod_linux.png	3. 권한 : 서랍 열쇠 규칙
17a_chmod_exit.png	3. 권한 : 서랍 열쇠 규칙
18_ps_logs.png	5. 첫 도시락 : hello-world와 ubuntu
18a_images_after_exit.png	5. 첫 도시락 : hello-world와 ubuntu
19_exec_vs_run.png	5. 첫 도시락 : hello-world와 ubuntu
20_docker_stats.png	5. 첫 도시락 : hello-world와 ubuntu
21_docker_build.png	6. 내 레시피 : Dockerfile과 이미지
22_port_mapping.png	7. 정문 번호 : 포트 매핑
23_browser_8080.png	7. 정문 번호 : 포트 매핑
24_browser_8081.png	7. 정문 번호 : 포트 매핑
25_connection_refused.png	7. 정문 번호 : 포트 매핑
26_docker_start_ok.png	7. 정문 번호 : 포트 매핑
27_bind_mount_before_after.png	8. 창문과 사물함 : 바인드 마운트, 볼륨

번호/이름	쓰인 장
30_not_a_git_repository.png	9. 일기장과 클라우드 : Git과 GitHub
32_volume_persist.png	8. 창문과 사물함 : 바인드 마운트, 볼륨
33_git_config.png	9. 일기장과 클라우드 : Git과 GitHub
34_git_remote.png	9. 일기장과 클라우드 : Git과 GitHub
35_vscode_github.png	9. 일기장과 클라우드 : Git과 GitHub
36_vscode_source_control.png	9. 일기장과 클라우드 : Git과 GitHub
36a_vscode_explorer.png	9. 일기장과 클라우드 : Git과 GitHub
40_compose_basic.png	10. 보너스 1~4 : Docker Compose와 환경 변수
41_compose_basic_browser.png	10. 보너스 1~4 : Docker Compose와 환경 변수
42_compose_no_config_fix.png	10. 보너스 1~4 : Docker Compose와 환경 변수
43_compose_down.png	10. 보너스 1~4 : Docker Compose와 환경 변수
44_compose_multi_up.png	10. 보너스 1~4 : Docker Compose와 환경 변수
45_compose_multi_network.png	10. 보너스 1~4 : Docker Compose와 환경 변수
47_compose_ops_logs.png	10. 보너스 1~4 : Docker Compose와 환경 변수
48_compose_ops_stop_start.png	10. 보너스 1~4 : Docker Compose와 환경 변수
49_compose_env_override.png	10. 보너스 1~4 : Docker Compose와 환경 변수
49a_compose_env_check.png	10. 보너스 1~4 : Docker Compose와 환경 변수
50_compose_env_cleanup.png	10. 보너스 1~4 : Docker Compose와 환경 변수
51_ssh_keygen.png	11. 보너스 5 : SSH 열쇠와 자물쇠
52_github_ssh_key_added.png	11. 보너스 5 : SSH 열쇠와 자물쇠
53_ssh_test.png	11. 보너스 5 : SSH 열쇠와 자물쇠
54_ssh_remote.png	11. 보너스 5 : SSH 열쇠와 자물쇠
55_ssh_push.png	11. 보너스 5 : SSH 열쇠와 자물쇠
56_port_conflict_error.png	12. 포트가 겹치면? 직접 일으켜 본 실험
57_port_conflict_diagnosis.png	12. 포트가 겹치면? 직접 일으켜 본 실험
57a_port_conflict_pid.png	12. 포트가 겹치면? 직접 일으켜 본 실험

모두 **57장**이 이 문서에 실려 있어요.